

A PROJECT REPORT

As partial requirement for the degree of Master Of Science (Computer Application)

ON

Online Clothe Shopping System [Wardrobe X]

Group No: 14

SUBMITTED BY : Mr. Yash Patel

University Exam No.115

UNDER GUIDANCE OF

Dr. Rachna Shukla

COMPUTER SCIENCE AND APPLICATION

DEPARTMENT,

Dolat-Usha Institute Of Applied Sciences And Dhiru-Sarla

Institute of Management & Commerce, Valsad

Affiliated with Veer Narmad South Gujarat University,

Surat

*** 2025-2026 ***

**DOLAT-USHA INSTITUTE OF APPLIED SCIENCES AND
DHIRU-SARLA INSTITUTE OF MANAGEMENT & COMMERCE, VALSAD.**



Computer Science and Application Department

CERTIFICATE

This is to certify that the project work is satisfactorily carried out on **Online Clothe Shopping System [Wardrobe x]** is a work done by **Mr. Yash Patel** in partial fulfillment of the requirement for the award of the degree of Master of Science (Computer Application) during the academic year 2025-2026 .I appreciate their hard work, honesty and discipline during the training for concerned project in our organization. The project work was carried out under our supervision and guidance of internal guide.

Dr. Nirav Desai

Project Guide

Project Certified

Dr. Snehal Joshi

I / C Principal and HOD

Date: _____

At M.SC (CA) Examination

Examiners:

1. _____

2. _____

Project Assessment Detail

Project Title: -Online Cloth Shopping System [Wardrobe X]

Week No.	Week From – Week To	Date of Report	Topic	Task Schedule for Next Week	Remarks of Internal Guide	Signature of Internal Guide
1	28/07/25 TO 02/08/25		Project Definition	Requirement Analysis		
2	04/08/25 TO 09/08/25		Requirement Analysis	System Design		
3	11/08/25 TO 16/08/25		System Design	System Design		
4	18/08/25 TO 23/08/25		System Design	Form Design		
5	25/08/25 TO 30/08/25		Form Design	Coding		
6	01/09/25 TO 06/09/25		Coding	Coding		
7	08/09/25 TO 13/09/25		Coding	Documentation		
8	15/09/25 TO 20/09/25		Documentation	---		

Acknowledgement

The successful completion of this documentation would not have been possible without the support and guidance of several individuals, to whom we express our sincere gratitude.

First and foremost, we would like to extend our heartfelt thanks to our guide, Dr. Nirav Desai, for his constant support, valuable suggestions, and guidance throughout every stage of this project. His encouragement and expert advice have been instrumental in helping us complete and present this work with precision.

We are also deeply grateful to our parents, who have always been our strongest pillars of support. Their motivation, patience, and unwavering belief in us have inspired and encouraged us throughout this journey. Without their love and constant support, completing this documentation would not have been possible.

Finally, we would like to express our sincere thanks to our Head of Department (HOD) and all the faculty members of our college. Their continuous support, guidance, and encouragement have played a significant role in shaping our knowledge and capabilities, enabling us to successfully complete this project.

We are truly thankful to everyone who contributed, directly or indirectly, to the successful completion of this work.

Yash Patel

INDEX

SR. No	Topics	Page No
1	Project Profile	
1.1	Project goal and objectives	
1.2	Scope of the project	
2	System Environment Hardware Software environment	
3	Platform Environment	
3.1	Front End Used	
3.2	Back End Used	
4	System Requirement Analysis	
5	System Design	
5.1	System Flow Diagram	
5.2	Data flow diagram	
5.3	ER diagram	
6	Data Dictionary	
7	Form Designing and coding	
8	Reports	
9	Testing	
10	Conclusion and Future Enhancement	
11	References/ Bibliography	

1.Project Profile

1. Project Goal:

- **Provide Trendy Styles:**

Offer a wide variety of fashionable clothing that aligns with the latest trends.

- **Ensure Quality and Affordability:**

Deliver high-quality products at prices that offer great value.

- **Enhance Customer Experience:**

Simplify shopping with a user-friendly interface and responsive customer support.

2. Target Audience:

- **Style Enthusiasts:**

Individuals who love keeping up with current fashion trends.

- **Value Seekers:**

Customers looking for stylish, high-quality clothing at budget-friendly prices.

- **Adults and Couples:**

Catering to men, women, and couples with a wide selection of casual, formal, and personalized clothing options.

2. System Environment

1. Hardware Environment:

- **Processor:** intel i5
- **RAM:** upto 8GB RAM ○ **Storage:** upto 512GB
- **Admin Module:**

- **Features:**

Manage Accounts: Add, update, delete user accounts. ○ **Manage**

Categories: Organize and update product categories. ○ **Product**

Management: Add, update, delete product details. ○ **Order**

Management: View, process, and update order statuses. ○ **View**

Reports: Generate and view reports on sales, user activities, and more.

- **User Module:**

- **Features:**

Profile Management: View and edit personal details like name, email, and password.

Product Browsing: Search and view product details.

Shopping Cart: Add or remove products, and view total prices. ○

Order History: View past orders with details like date, products, and status.

3.Platform Environment

1. Front End:

Technologies:

React.js – For building dynamic and responsive user interfaces.

HTML, TailwindCSS, JavaScript – Core web technologies for structure and styling.

2. Back End:

○ Technologies:

Node.js – Server-side runtime environment.

Express.js – Web framework for building APIs and handling requests.

3. Database Design:

○ Technology:

MongoDB – NoSQL database to store users, courses, lessons, and results.

4. System Requirement Analysis

1. Index Page

- **Functionality:**
 - Introduce the website's purpose and features. o Include navigation links to Registration and Login pages.
 - Display a logo or brief introduction.
- **Goal:**
 - Create a positive first impression.
 - Encourage user interaction.

2. Registration Page

- **Functionality:**
 - Form to collect user details: Name, Email, Password. o Validation for input fields. o Submit data to the database.
- **Goal:**
 - Allow users to create an account for future interaction.

3. Login Page :

- **Functionality:**
 - Form with fields: Email and Password. o Include a "Forgot Password" option. o Validate and authenticate user credentials.
- **Goal:**
 - Provide secure access to the website for registered users.

4. Product Page :

- **Functionality:**
 - Display products with details: Image, Price, and Description. o Allow users to add products to the cart.
- **Goal:**
 - Enable users to browse and choose products.

5. Shopping Cart :

- **Functionality:**
 - Show selected products with details: Name, Quantity, Price. o Options to update quantities, remove items, or proceed to checkout.
- **Goal:**
 - Simplify the purchase process by summarizing selected items.

Additional Features :

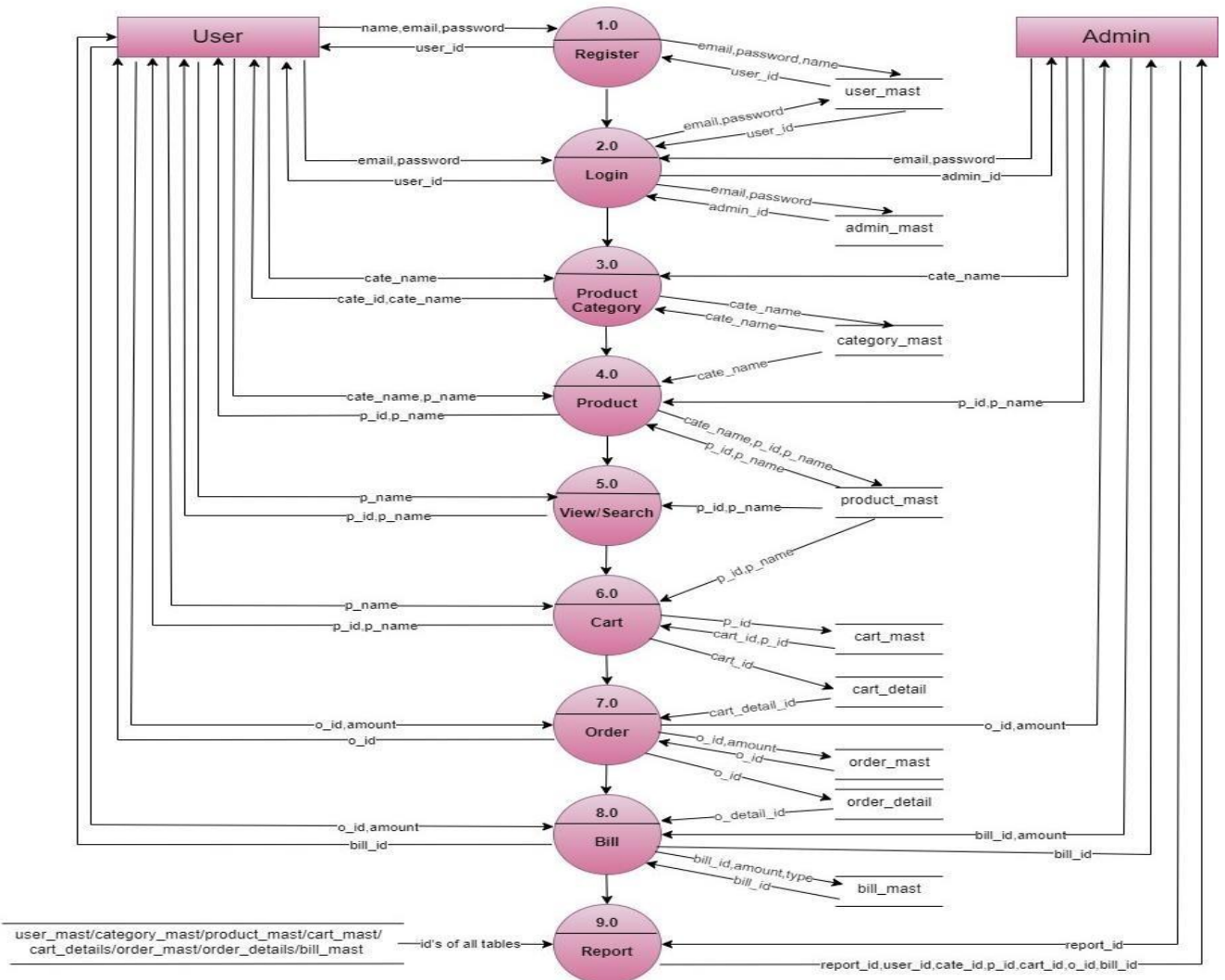
- **Password Recovery:** Allow users to reset forgotten passwords.
- **Responsive Design:** Ensure mobile compatibility.
- **Admin Panel:**
 - o Manage users and product details.
 - o Monitor transactions.

5. System Design

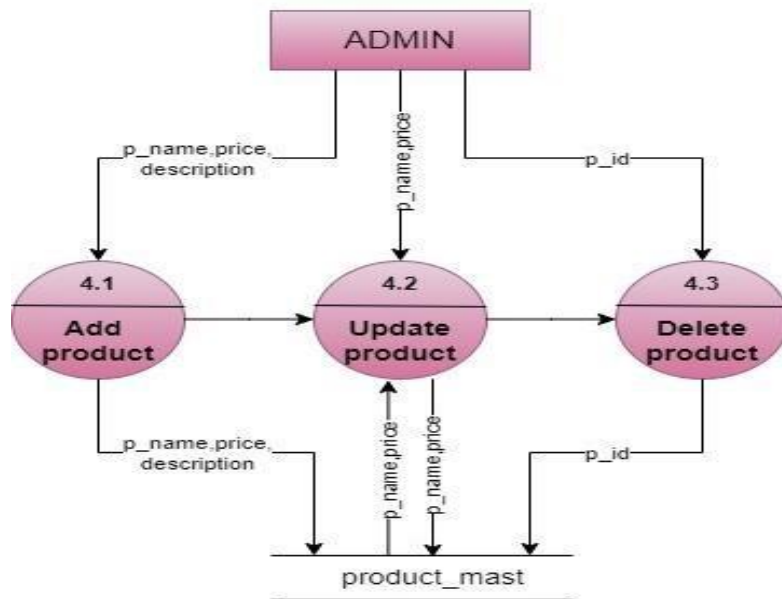
Data Flow Diagram [DFD] 0 Level :



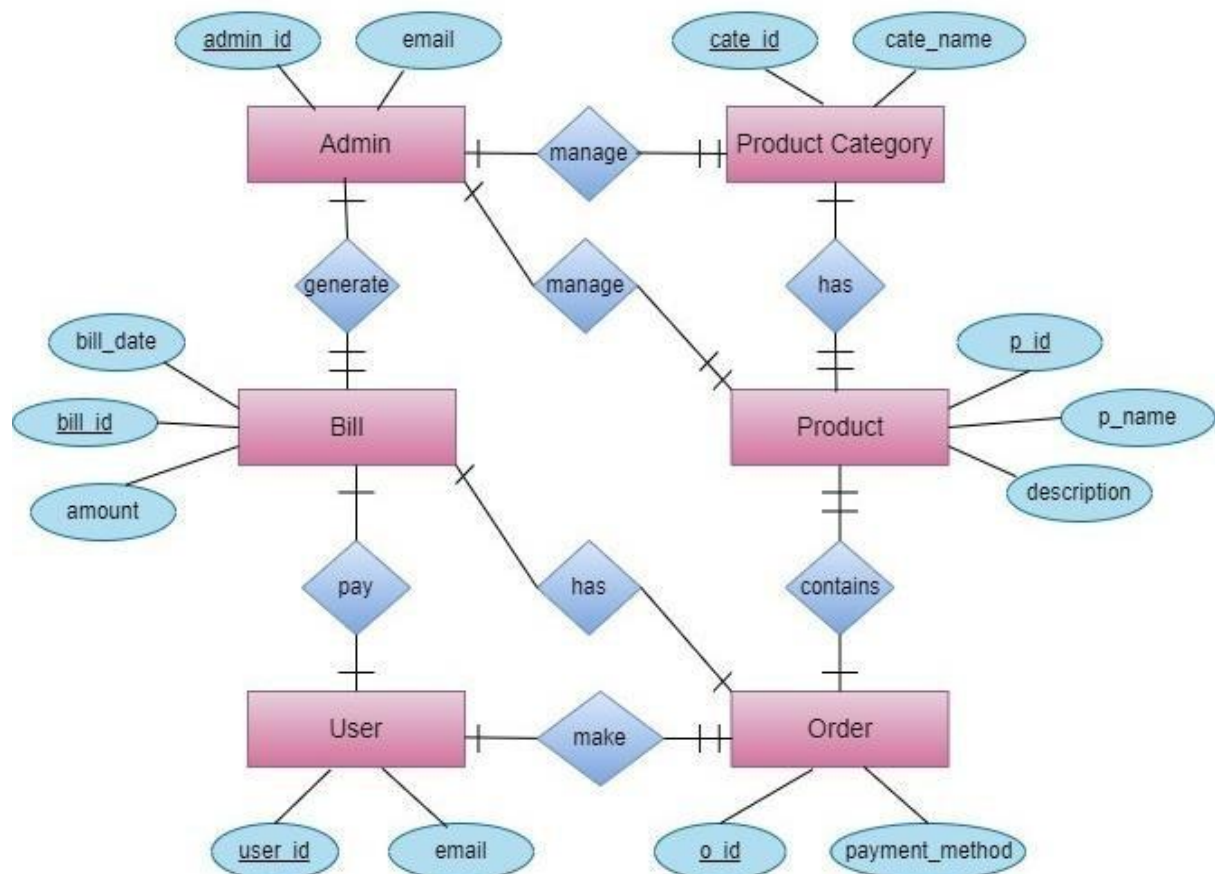
Data Flow Diagram [DFD] 1st Level :



Data Flow Diagram [DFD] 2nd Level :



ER Diagram :



6. Data Dictionary

Admin_mast :

Name	Type	Null	Key
admin_id	String	No	Primary key
name	String	No	
email	String	No	Unique
password	String	No	
Role	String	No	Default: admin

User :

Field	Type	Null	Key
_id	String	No	Primary
name	String	No	
email	String	No	Unique
phone	String	No	
password	String	No	

Category_mast :

Field	Type	Null	Key
cate_id	String	No	Primary
catename	String	No	

Product_mast :

Name	Type	Null	Key
p_id	String	No	Primary
p_name	String	No	-
description	String	No	-
details	String	No	-
price	String	No	-
img	String	No	--
stock	String	No	

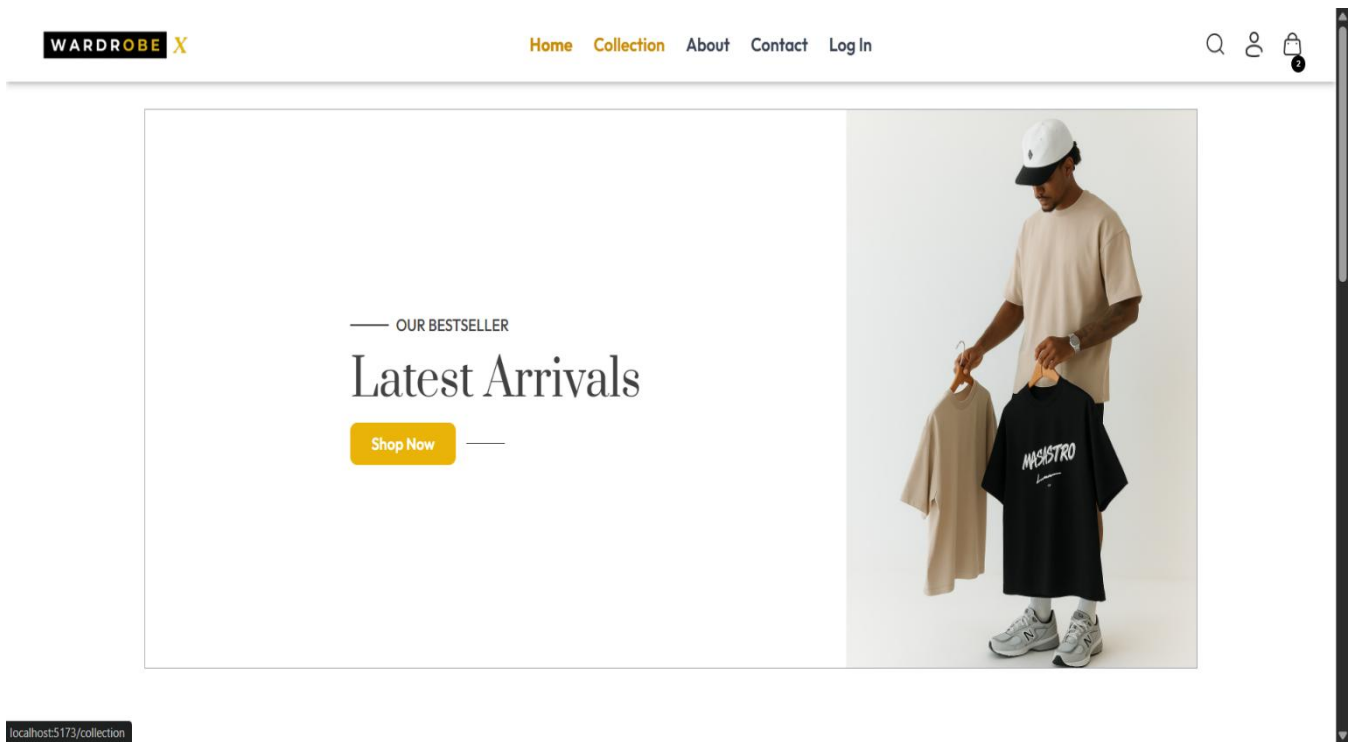
Cart_mast :

Field Name	Type	Null	Key
cart_id	String	Null	
user_id	String	No	(user_id) References user_mast (user_id)
created_at	timestamp	No	current_timestamp()

Order_details :

Name	Type	Null	Key
Order_id	String	No	Primary
CustomerName	String	No	-
email	String	No	-
phone	String	No	-
address	String	No	-
city	String	No	-
zip	String	No	-
state	String	No	-
PaymentMethod	String	No	-
Status	String	No	-
TotalAmount	String	No	-

User Side :



Code :

ShopContext :

```
import React, { createContext, useState, useEffect } from "react";
import api from "../api/api";
```

```
export const ShopContext = createContext();
```

```
const ShopContextProvider = ({ children }) => {
  const [products, setProducts] = useState([]);
  const [cartItems, setCartItems] = useState({});
  const [cartCount, setCartCount] = useState(0);
  const currency = "₹";
  const delivery_fee = 60;
```

```

// Fetch products
useEffect(() => {
  const fetchProducts = async () => {
    try {
      const res = await api.get("/api/product");
      setProducts(res.data.products || []);
    } catch (err) {
      console.error(err);
    }
  };
  fetchProducts();
}, []);

// Fetch cart from backend
const fetchCart = async () => {
  const token = localStorage.getItem("token");
  if (token) {
    try {
      const res = await api.get("/api/cart", {
        headers: { Authorization: `Bearer ${token}` },
      });
      const items = {};
      res.data.items.forEach((item) => {
        if (!items[item.product._id]) items[item.product._id] = {};
        items[item.product._id][item.size] = item.quantity;
      });
      setCartItems(items);
    } catch (err) {
      console.error("Error fetching cart:", err);
    }
  } else {
    // Guest cart
    const guestCart = JSON.parse(localStorage.getItem("guestCart")) || {};
    setCartItems(guestCart);
  }
};

```



```

// Calculate cart count whenever cartItems change
useEffect(() => {
  const count = Object.values(cartItems).reduce((acc, sizes) => {
    return acc + Object.values(sizes).reduce((a, q) => a + q, 0);
  }, 0);
  setCartCount(count);
}, [cartItems]);

// Fetch cart once on mount
useEffect(() => {
  fetchCart();
}, []);

// Add product to cart
const addToCart = async (productId, size, quantity = 1) => {
  const token = localStorage.getItem("token");

  if (token) {
    // Logged-in: use backend
    try {
      await api.post(
        "/api/cart/add",
        { productId, size, quantity },
        { headers: { Authorization: `Bearer ${token}` } }
      );
      fetchCart(); // refresh cart and count
    } catch (err) {
      console.error("Error adding to cart:", err);
    }
  } else {
    // Guest: store in localStorage
    const guestCart = JSON.parse(localStorage.getItem("guestCart")) || {};
    if (!guestCart[productId]) guestCart[productId] = {};
    guestCart[productId][size] = (guestCart[productId][size] || 0) + quantity;
    localStorage.setItem("guestCart", JSON.stringify(guestCart));
  }
}

```

```
    setCartItems(guestCart); // update state to refresh Navbar count
  }
};
```

```
const value = {
  products,
  cartItems,
  setCartItems,
  cartCount,
  fetchCart, // expose fetchCart so components can refresh it
  currency,
  delivery_fee,
  addToCart,
};
```

```
return <ShopContext.Provider value={value}>{children}</ShopContext.Provider>;
};
```

```
export default ShopContextProvider;
```

Components :

Hero.jsx :

```
import React from 'react';
import { assets } from '../assets/frontend_assets/assets';
import { Link } from "react-router-dom";

const Hero = () => {
  return (
    <section className="flex flex-col sm:flex-row border-b border-gray-300 rounded-lg ">
      {/* Left side */}
      <div className="w-full sm:w-2/3 flex items-center justify-center py-10 sm:py-0 px-6">
        <div className="text-gray-800 space-y-6 max-w-xl">
          {/* Small heading */}
          <div className="flex items-center gap-3">
            <span className="w-10 h-[2px] bg-gray-800"></span>
            <p className="font-medium text-sm md:text-base">OUR BESTSELLER</p>
          </div>

          {/* Main heading */}
          <h1 className="text-3xl lg:text-5xl font-extrabold leading-tight">
            Latest Arrivals
          </h1>

          {/* CTA */}
          <div className="flex items-center gap-4 mt-4">
            <Link
              to="/collection"
              className="px-6 py-2 bg-yellow-500 text-white font-semibold rounded-lg hover:bg-yellow-600 transition"
            >
              Shop Now
            </Link>
            <span className="w-10 h-[1px] bg-gray-800"></span>
          </div>
        </div>
      </div>
    </section>
  );
};
```

```
</div>
```

```
{/* Right side */}
```

```
<div className="w-full sm:w-1/3 flex justify-start items-center p-0">
```

```
<img
```

```
  className="w-80 sm:w-96 object-cover rounded-lg shadow-2xl hover:shadow-3xl transform  
  hover:scale-105 transition-all duration-500"
```

```
  src={assets.hero_img}
```

```
  alt="Hero"
```

```
</div>
```

```
</section>
```

```
);
```

```
};
```

```
export default Hero;
```

Latestcollection.jsx :

```
import React, { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import api from '../api/api';
import Title from './Title';

const LatestCollection = () => {
  const [latestProducts, setLatestProducts] = useState([]);
  const backendURL = "http://localhost:3000"; // Backend base URL

  useEffect(() => {
    const fetchProducts = async () => {
      try {
        const res = await api.get('/api/product');
        if (res.data.success && res.data.products) {
          setLatestProducts(res.data.products.slice(0, 8));
        }
      } catch (err) {
        console.error('Error fetching products:', err);
      }
    };
    fetchProducts();
  }, []);

  return (
    <section className="my-12 px-5">
      { /* Heading */ }
      <div className="text-center py-8">
        <Title text1="LATEST" text2="COLLECTIONS" />
        <p className="w-3/4 mx-auto text-gray-600 text-sm sm:text-base mt-3">
          Step into a season of bold expression and effortless elegance. Our newest collection
          celebrates individuality — crafted for those who dress with intention and live with flair.
        </p>
      </div>
      <div className="grid grid-cols-2 sm:grid-cols-3 md:grid-cols-4 gap-6">
```

```

    {latestProducts.map((item, index) => {
      const productImage = Array.isArray(item.image) ? item.image : [item.image];

      return (
        <Link
          key={index}
          to={` /product/${item._id}`}
          className="flex flex-col items-center bg-white shadow-md rounded-2xl overflow-
hidden hover:shadow-xl transition-transform duration-300 hover:-translate-y-1"
        >
          <div className="w-full aspect-square bg-gray-100 flex items-center justify-center
overflow-hidden">
            <img
              src={
                productImage[0]
                ? `${backendURL}${productImage[0]}`
                : "https://via.placeholder.com/300x300?text=No+Image"
              }
              alt={item.name}
              className="w-full h-full object-cover transition-transform duration-300 hover:scale-
105"
            />
          </div>

          <div className="p-3 text-center">
            <p className="text-sm sm:text-base font-semibold truncate">{item.name}</p>
            <p className="text-gray-600 text-xs sm:text-sm">₹{item.price}</p>
          </div>
        </Link>
      );
    })}
  </div>
</section>

);
};
export default LatestCollection;

```

Navbar.jsx :

```
import React, { useState, useEffect, useContext } from "react";
import { Link, NavLink, useNavigate } from "react-router-dom";
import logo from "../assets/Images/logo.png";
import { assets } from "../assets/frontend_assets/assets";
import { ShopContext } from "../context/ShopContext";

const Navbar = () => {
  const [isOpen, setIsOpen] = useState(false);
  const navigate = useNavigate();
  const { cartCount } = useContext(ShopContext); // reactive cart count

  // Login state
  const [isLoggedIn, setIsLoggedIn] = useState(
    localStorage.getItem("isLoggedIn") === "true"
  );

  // Logout handler
  const handleLogout = () => {
    localStorage.removeItem("isLoggedIn");
    setIsLoggedIn(false); // update state immediately
    navigate("/"); // redirect to home
  };

  // Update login state if localStorage changes (cross-tab)
  useEffect(() => {
    const updateLogin = () => {
      setIsLoggedIn(localStorage.getItem("isLoggedIn") === "true");
    };
    window.addEventListener("storage", updateLogin);
    return () => window.removeEventListener("storage", updateLogin);
  }, []);

  // Menu items
  const menuItems = [
```

```

    { path: "/", label: "Home" },
    { path: "/collection", label: "Collection" },
    { path: "/about", label: "About" },
    { path: "/contact", label: "Contact" },
    !isLoggedIn && { path: "/login", label: "Log In" },
  ].filter(Boolean);

```

```

return (
  <nav className="fixed top-0 left-0 w-full bg-white shadow-md px-5 sm:px-10 py-5 flex
items-center justify-between z-50">

```

```

    {/* Logo */}
    <Link to="/">
      <img src={logo} alt="Logo" className="w-44" />
    </Link>

```

```

    {/* Desktop Menu */}
    <ul className="hidden sm:flex gap-6 text-lg text-gray-700 font-semibold">
      {menuItems.map((item, idx) => (
        <NavLink
          key={idx}
          to={item.path}
          end
          className={({ isActive }) =>
            `group flex flex-col items-center gap-1 transition-colors duration-300 ${
              isActive
                ? "text-yellow-600 font-bold"
                : "text-gray-700 hover:text-yellow-600"
            }`
          }
        >
          {item.label}
        </NavLink>
      ))}

```

```

    {/* Logout button */}
    {isLoggedIn && (

```



```

    <button
      onClick={handleLogout}
      className="text-gray-700 hover:text-yellow-600"
    >
      Log Out
    </button>
  )}
</ul>

```

```

    { /* Right section: Cart and Profile */ }
<div className="flex items-center gap-6">
  { /* Profile Dropdown - only if logged in */ }
  { isLoggedIn && (
    <div className="group relative">
      <img
        src={assets.profile_icon}
        alt="Profile"
        className="w-5 cursor-pointer"
      />
      <div className="group-hover:block hidden absolute dropdown-menu right-0 pt-3">
        <div className="flex flex-col gap-2 w-36 py-3 px-5 bg-slate-100 text-gray-500 rounded">
          <Link to="/myprofile">
            <p className="cursor-pointer hover:text-black">My Profile</p>
          </Link>
          <Link to="/myorders">
            <p className="cursor-pointer hover:text-black">Orders</p>
          </Link>
        </div>
      </div>
    </div>
  ) }

```

```

    { /* Cart */ }
    <Link to="/cart" className="relative">
      <img src={assets.cart_icon} alt="Cart" className="w-5" />
      { cartCount > 0 && (

```

```

    <span className="absolute -top-2 -right-2 bg-black text-white text-[10px] w-4 h-4 rounded-
full flex items-center justify-center">
      {cartCount}
    </span>
  )}
</Link>
</div>

```

```

  { /* Mobile menu button */ }
  <div className="sm:hidden">
    <button
      onClick={() => setIsOpen(true)}
      className="relative w-8 h-8 flex items-center justify-center group"
      aria-label="Open menu"
    >
      <span className="absolute top-2 w-6 h-[1px] bg-black transition-all duration-300 group-
hover:top-1/2 group-hover:rotate-90" />
      <span className="absolute bottom-2 w-6 h-[1px] bg-black transition-all duration-300
group-hover:bottom-1/2" />
    </button>
  </div>
  <div
    className={`fixed top-0 right-0 h-full w-64 bg-white shadow-lg transform ${
      isOpen ? "translate-x-0" : "translate-x-full"
    } transition-transform duration-300 ease-in-out sm:hidden z-50`}
  >
    <div className="flex justify-end p-4">
      <button onClick={() => setIsOpen(false)}>✕</button>
    </div>
    <ul className="flex flex-col items-start gap-6 p-6 mt-2 font-semibold">
      {menuItems.map((item, idx) => (
        <NavLink
          key={idx}
          to={item.path}
          end
          onClick={() => setIsOpen(false)}

```

```

      className={({ isActive }) =>
        `text-lg transition-colors duration-300 ${
          isActive
            ? "text-yellow-600 font-bold"
            : "text-gray-700 hover:text-yellow-600"
        }`
      }
    >
      {item.label}
    </NavLink>
  )}
)

{isLoggedIn && (
  <button
    onClick={() => {
      handleLogout();
      setIsOpen(false);
    }}
    className="text-lg text-gray-700 hover:text-yellow-600"
  >
    Log Out
  </button>
)}
</ul>
</div>

{isOpen && (
  <div
    className="fixed inset-0 bg-black bg-opacity-25 z-40 sm:hidden"
    onClick={() => setIsOpen(false)}
  ></div>
)}
</nav>
);
};

export default Navbar;

```

Policy.jsx :

```
import React from "react";
import { RefreshCw, ShieldCheck, Headphones } from "lucide-react";
const PolicySection = () => {
  const policies = [
    {
      icon: <RefreshCw size={40} className="text-black" />,
      title: "Easy Exchange Policy",
      description: "We offer hassle free exchange policy",
    },
    {
      icon: <ShieldCheck size={40} className="text-black" />,
      title: "7 Days Return Policy",
      description: "We provide 7 days free return policy",
    },
    {
      icon: <Headphones size={40} className="text-black" />,
      title: "Best Customer Support",
      description: "We provide 24/7 customer support",
    },
  ];
  return (
    <section className="w-full py-16 bg-white">
      <div className="max-w-6xl mx-auto px-5 grid grid-cols-1 sm:grid-cols-3 gap-12 text-center">
        {policies.map((item, index) => (
          <div key={index} className="flex flex-col items-center">
            <div className="mb-4">{item.icon}</div>
            <h3 className="text-lg font-semibold mb-2">{item.title}</h3>
            <p className="text-gray-500 text-sm">{item.description}</p>
          </div>
        ))}
      </div>
    </section>
  );
};
export default PolicySection;
```

ProductItem.jsx :

```
import React, { useContext } from 'react';
import { ShopContext } from '../context/ShopContext';
import { Link } from 'react-router-dom';

const ProductItem = ({ id, image, name, price }) => {
  const { currency } = useContext(ShopContext);

  return (
    <Link className='text-gray-700 cursor-pointer' to={` /api/product/${id}`} >
      <div className='overflow-hidden'>
        <img
          className='hover:scale-110 transition ease-in-out w-full h-48 object-cover rounded-lg'
          src={image} // single string
          alt={name}
        />
      </div>
      <p className='py-3 pb-1 text-sm'>{name}</p>
      <p className='text-sm font-medium'>{currency} {price}</p>
    </Link>
  );
};

export default ProductItem;
```

Title.jsx :

```
import React from 'react'
```

```
const Title = ({text1, text2}) => {  
  return (  
    <div className='inline-flex gap-2 items-center mb3'>  
      <p className='text-gray-500'>{text1} <span className='text-gray-700 font-  
medium'>{text2}</span></p>  
      <p className='w-8 sm:w-12 h-[1px] sm:-[2px] bg-gray-700 ' ></p>  
    </div>  
  )  
}
```

```
export default Title
```

Footer.jsx :

```
import React from 'react';
import logo from '../assets/Images/logo.png';

const Footer = () => {
  return (
    <footer className="w-full bg-gray-50 text-gray-700 mt-20 pb-10">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div className="flex flex-col sm:flex-row justify-between items-start gap-10">

          {/* Left section - Logo + About */}
          <div className="sm:w-2/3">
            <img src={logo} className="w-44 h-16 object-contain mb-5" alt="Logo" />
            <p className="w-full md:w-2/3 text-gray-600">
              Discover the latest trends in fashion and shop your favorite styles
              with ease. Quality clothing, best prices, and fast delivery.
            </p>
          </div>

          {/* Middle section - Company Links */}
          <div className="sm:w-1/6">
            <p className="text-xl font-medium mb-5">COMPANY</p>
            <ul className="flex flex-col gap-2 text-gray-600">
              <li>HOME</li>
              <li>ABOUT US</li>
              <li>DELIVERY</li>
            </ul>
          </div>

          {/* Right section - Get in Touch */}
          <div className="sm:w-1/3">
            <h3 className="text-lg font-semibold mb-3">GET IN TOUCH</h3>
            <ul className="flex flex-col gap-1 text-gray-600">
              <li>+91 8789655872</li>
              <li>wordobex2@gmail.com</li>
            </ul>
          </div>
        </div>
      </div>
    </footer>
  );
}
```

</div>

</div>

{/* Bottom copyright */}

<div className="mt-10 border-t border-gray-300 pt-5">

<p className="text-sm text-center text-gray-500">

Copyright 2025 @ woredrobex.com - All Rights Reserved.

</p>

</div>

</div>

</footer>

);

};

export default Footer;

AdminSideBar.jsx :

```
import React from "react";
import { NavLink, useNavigate } from "react-router-dom";
import { Logout, PackagePlus, Eye, ShoppingBag } from "lucide-react";

const AdminSidebar = () => {
  const navigate = useNavigate();

  const handleLogout = () => {
    localStorage.removeItem("adminToken");
    localStorage.setItem("isLoggedIn", "false");
    window.dispatchEvent(new Event("storage"));
    navigate("/");
  };

  const menuItems = [
    { name: "Dashboard", path: "/admin", icon: <PackagePlus size={18} /> },
    { name: "Add Product", path: "/admin/add-product", icon: <PackagePlus size={18} /> },
    { name: "View Products", path: "/admin/view-products", icon: <Eye size={18} /> },
    { name: "Customer Orders", path: "/admin/view-orders", icon: <ShoppingBag size={18} /> },
  ];

  return (
    <aside className="w-64 h-screen bg-gradient-to-b from-gray-900 via-gray-800 to-gray-900 text-gray-200 flex flex-col shadow-xl sticky top-0 border-r border-gray-700">
      <div className="p-5 border-b border-gray-700">
        <h1 className="text-2xl font-bold text-center text-white tracking-wide">
          Admin Panel
        </h1>
      </div>

      <div className="p-5">
        <h2>Menu</h2>
        <ul>
          {menuItems.map((item) => (
            <li>
              <NavLink
                key={item.path}
                to={item.path}
                className="text-gray-200 flex items-center gap-2 p-2 rounded">
                {item.icon}
                {item.name}
              </NavLink>
            </li>
          ))}
        </ul>
      </div>
    </aside>
  );
};
```

```

      key={item.name}
      to={item.path}
      className={({ isActive }) =>
        `flex items-center gap-3 px-4 py-2 rounded-lg font-medium transition-all duration-200
$ {
      isActive
        ? "bg-blue-600 text-white shadow-md"
        : "hover:bg-gray-700 hover:text-white text-gray-300"
      }`
    }
  >
    {item.icon}
    <span>{item.name}</span>
  </NavLink>
)}
</nav>

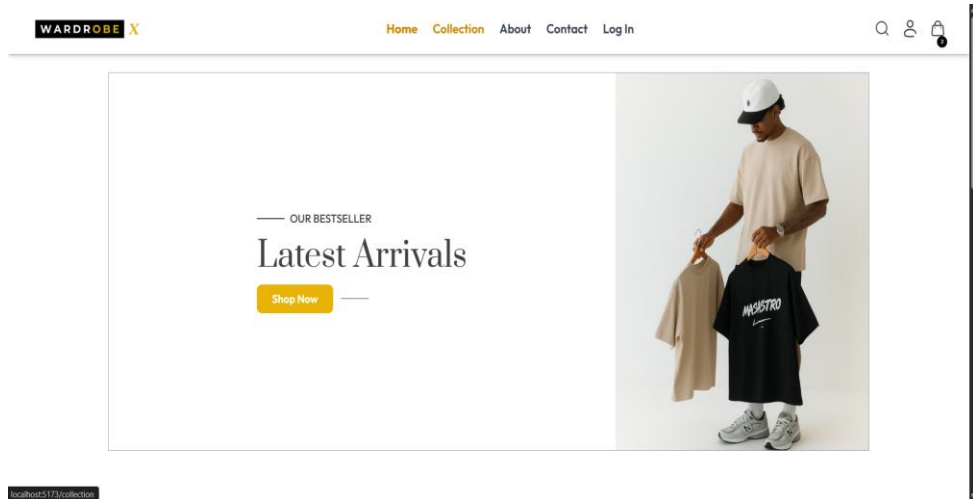
{/* Logout Button */}
<div className="p-4 border-t border-gray-700">
  <button
    onClick={handleLogout}
    className="w-full flex items-center justify-center gap-2 bg-red-600 hover:bg-red-700
text-white font-medium py-2 rounded-lg transition-all duration-200"
  >
    <LogOut size={18} />
    Logout
  </button>
</div>
</aside>
);
};

export default AdminSidebar;

```

User Pages :

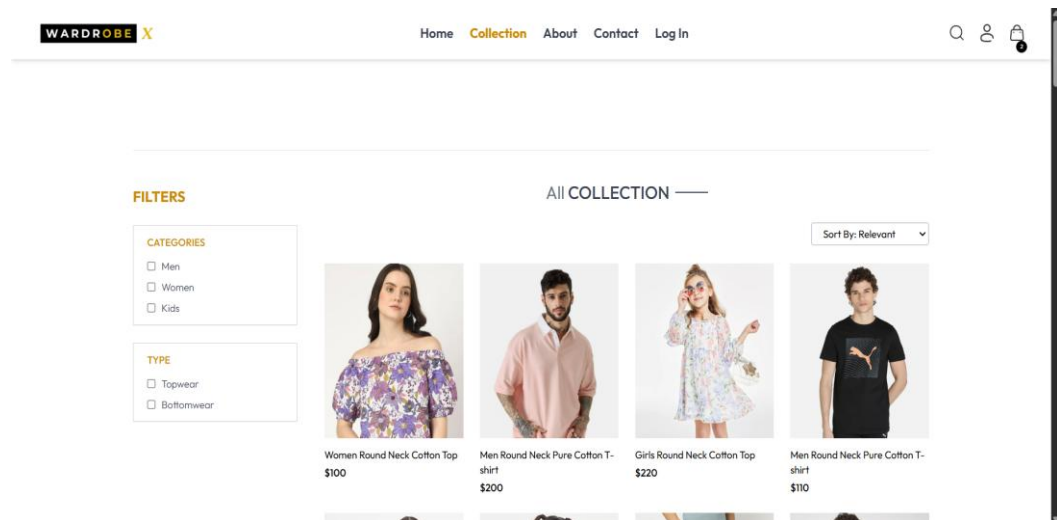
Home.jsx :



```
import React from 'react'
import Navbar from '../components/Navbar';
import Hero from '../components/Hero';
import LatestCollection from '../components/LatestCollection';
import PolicySection from '../components/Policy';
const home = () => {
  return (
    <main className="pt-24 px-5">
      <div>
        <Navbar />
      <Hero />
      <LatestCollection />
      <PolicySection/>
    </div>
    </main>
  );
}
```

export default home;

Collection.jsx :



```
import React, { useEffect, useState } from "react";
```

```
import { Link } from "react-router-dom";
```

```
import api from "../api/api"; // your Axios instance
```

```
import { assets } from "../assets/frontend_assets/assets";
```

```
import Title from "../components/Title";
```

```
import PolicySection from "../components/Policy";
```

```
const Collection = () => {
```

```
  const [products, setProducts] = useState([]);
```

```
  const [showFilter, setShowFilter] = useState(false);
```

```
  const [filterProduct, setFilterProduct] = useState([]);
```

```
  const [category, setCategory] = useState([]);
```

```
  const [subCategory, setSubCategory] = useState([]);
```

```
  const [sortOption, setSortOption] = useState("relevant");
```

```
  const [searchText, setSearchText] = useState("");
```

```
  const backendURL = "http://localhost:3000"; // backend base URL
```

```
  // Fetch products
```

```

useEffect(() => {
  const fetchProducts = async () => {
    try {
      const res = await api.get("/api/product");
      setProducts(res.data.products || []);
    } catch (err) {
      console.error("Error fetching products:", err);
    }
  };
  fetchProducts();
}, []);

```

```

const toggleCategory = (e) => {
  const value = e.target.value;
  setCategory((prev) =>
    prev.includes(value) ? prev.filter((item) => item !== value) : [...prev, value]
  );
};

```

```

const toggleSubCategory = (e) => {
  const value = e.target.value;
  setSubCategory((prev) =>
    prev.includes(value) ? prev.filter((item) => item !== value) : [...prev, value]
  );
};

```

// Filter and sort products

```

useEffect(() => {
  let filtered = [...products];
  const normalizeText = (text) => text.toLowerCase().replace(/[-_ \s]/g, "");

  if (searchText.trim() !== "") {
    const normalizedSearch = normalizeText(searchText);
    filtered = filtered.filter((item) =>
      normalizeText(item.name).includes(normalizedSearch));
  }
}

```

```

    if (searchText.trim() === "") {
      if (category.length > 0) filtered = filtered.filter((item) =>
category.includes(item.category));
      if (subCategory.length > 0) filtered = filtered.filter((item) =>
subCategory.includes(item.subCategory));
    }

    if (sortOption === "low-high") filtered.sort((a, b) => a.price - b.price);
    if (sortOption === "high-low") filtered.sort((a, b) => b.price - a.price);

    setFilterProduct(filtered);
  }, [products, category, subCategory, sortOption, searchText]);

  return (
    <div className="px-5">
      <main className="pt-24">
        { /* Search Bar */ }
        <div className="mb-6 flex justify-center">
          <div className="inline-flex items-center justify-center border border-gray-
400 px-4 py-2 rounded-3xl bg-white w-full max-w-md">
            <input
              type="text"
              value={searchText}
              onChange={(e) => setSearchText(e.target.value)}
              placeholder="Search products..."
              className="flex-1 outline-none px-2 py-1 text-sm"
            />
            {searchText === "" ? (
              <img className="w-4 ml-2" src={assets.search_icon} alt="search" />
            ) : (
              <button type="button" onClick={() => setSearchText("")}
className="ml-2 focus:outline-none">
                <img className="w-4" src={assets.cross_icon} alt="clear" />
              </button>
            )}
          </div>
        </div>
      </main>
    </div>
  );

```

</div>

</div>

<div className="flex flex-col sm:flex-row gap-10 border-t py-10">

 {/* Left Filters */}

 <div className="min-w-60">

 <p

 className="my-2 text-xl flex items-center cursor-pointer gap-2 text-yellow-600 font-bold"

 onClick={() => setShowFilter(!showFilter)}

 >

 FILTERS

 <img

 className={`h-3 sm:hidden \${showFilter ? "rotate-90" : ""}`}

 src={assets.dropdown_icon}

 alt=""

 />

 </p>

 <div className={`border border-gray-300 pl-5 py-3 mt-6 shadow-sm my-6 \${showFilter ? "" : "hidden"} sm:block`} >

 <p className="mb-3 text-sm font-medium text-yellow-600">CATEGORIES</p>

 <div className="flex flex-col gap-2 text-sm font-light text-gray-700">

 <label className="flex gap-2 cursor-pointer">

 <input type="checkbox" value="Men" onChange={toggleCategory} />

Men

 </label>

 <label className="flex gap-2 cursor-pointer">

 <input type="checkbox" value="Women" onChange={toggleCategory} />

/> Women

 </label>

 <label className="flex gap-2 cursor-pointer">

 <input type="checkbox" value="Kids" onChange={toggleCategory} />

Kids

 </label>

</div>

</div>

<div className={`border border-gray-300 pl-5 py-3 mt-6 shadow-sm my-5
\${showFilter ? "" : "hidden"} sm:block`}>

<p className="mb-3 text-sm font-medium text-yellow-600">TYPE</p>

<div className="flex flex-col gap-2 text-sm font-light text-gray-700">

<label className="flex gap-2 cursor-pointer">

<input type="checkbox" value="Topwear"

onChange={toggleSubCategory} /> Topwear

</label>

<label className="flex gap-2 cursor-pointer">

<input type="checkbox" value="Bottomwear"

onChange={toggleSubCategory} /> Bottomwear

</label>

</div>

</div>

</div>

{/* Right Side */}

<div className="flex-1">

<div className="flex justify-center text-base sm:text-2xl mb-6">

<Title text1="All" text2="COLLECTION" />

</div>

<div className="flex justify-end mb-6">

<select

className="border-2 border-gray-300 text-sm px-3 py-1 rounded"

value={sortOption}

onChange={(e) => setSortOption(e.target.value)}

>

<option value="relevant">Sort By: Relevant</option>

<option value="low-high">Sort By: Low to High</option>

<option value="high-low">Sort By: High to Low</option>

</select>

</div>


```

    { /* Product Grid */}
    <div className="grid grid-cols-2 md:grid-cols-3 lg:grid-cols-4 gap-6">
      {filterProduct.map((item, index) => {
        const productImage = Array.isArray(item.image) ? item.image :
[item.image];

        return (
          <Link
            to={` /api/product/${item._id}`}
            key={index}
            className="flex flex-col items-center bg-white shadow-md rounded-
2xl overflow-hidden hover:shadow-lg transition-all"
          >
            <div className="w-full h-56 sm:h-60 md:h-64 lg:h-72 overflow-
hidden">
              <img
                src={productImage[0] ? `${backendURL}${productImage[0]}` :
"https://via.placeholder.com/150"}
                alt={item.name}
                className="w-full h-full object-cover object-center transition-
transform duration-300 hover:scale-105"
              />
            </div>

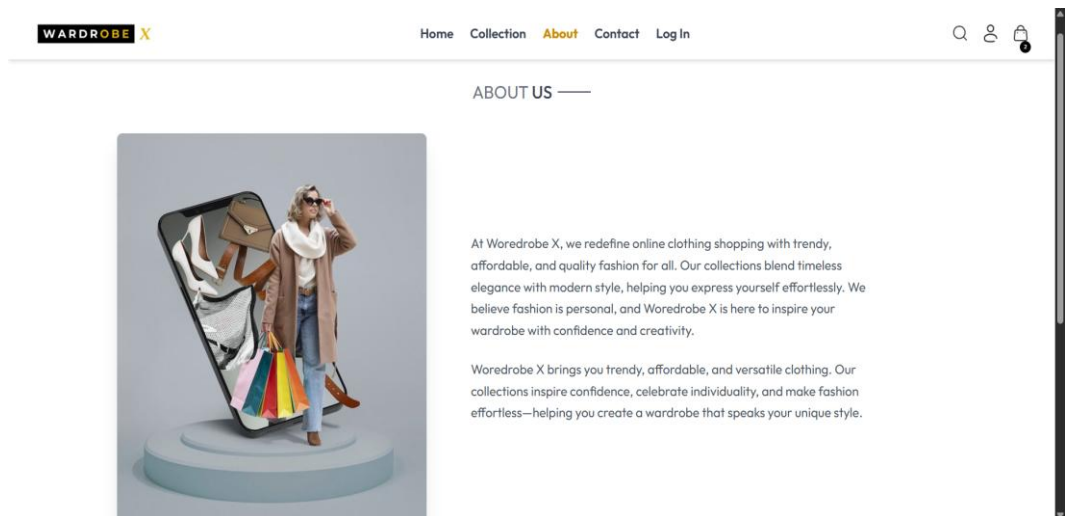
            <div className="p-3 text-center">
              <p className="text-sm sm:text-base font-
semibold">{item.name}</p>
              <p className="text-gray-600 text-xs sm:text-sm">₹{item.price}</p>
            </div>
          </Link>
        );
      })}
    </div>
  </div>
</div>

```

```
        <PolicySection />
    </main>
</div>
);
};

export default Collection;
```

About.jsx :



```
import React from 'react'
import Title from '../components/Title'
import { assets } from '../assets/frontend_assets/assets'
import { Link } from 'react-router-dom'
import PolicySection from '../components/Policy'

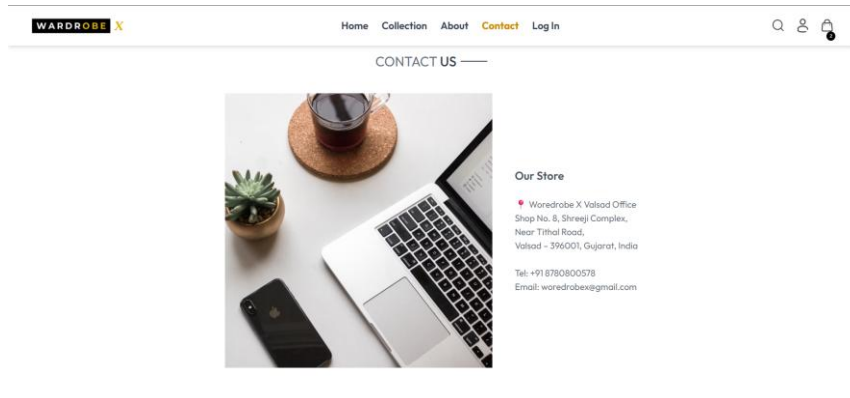
const About = () => {
  return (
    <main className="pt-24 px-5 bg-gray-50 min-h-screen">
      { /* Section Title */ }
      <div className="text-center pt-12">
        <Title text1="ABOUT" text2="US" />
        <p className="mt-3 text-gray-500 max-w-xl mx-auto text-sm sm:text-base">
          Discover our story, mission, and vision. At Wardrobe X, fashion meets style
and individuality.
        </p>
      </div>
      <section className="my-16 flex flex-col md:flex-row items-center justify-
center gap-12 max-w-6xl mx-auto">
        { /* Image */ }
        <div className="md:w-1/2 w-full">
          <img
```

```

        className="w-full rounded-2xl shadow-2xl hover:shadow-3xl transition-
shadow duration-500 transform hover:scale-105"
        src={assets.about_img}
        alt="About us"
      />
    </div>
    <div className="md:w-1/2 w-full flex flex-col justify-center gap-6 text-gray-
700">
      <p className="text-lg sm:text-xl leading-relaxed">
        At <span className="font-semibold text-gray-900">Wardrobe X</span>,
        we redefine online clothing shopping with trendy, affordable, and high-quality fashion
        for everyone. Our collections blend timeless elegance with modern style, helping you
        express yourself effortlessly.
      </p>
      <p className="text-lg sm:text-xl leading-relaxed">
        We believe fashion is personal, and <span className="font-semibold text-
gray-900">Wardrobe X</span> is here to inspire your wardrobe with confidence and
        creativity. Explore versatile, stylish pieces that celebrate individuality and make fashion
        effortless.
      </p>
      <div className="mt-6">
        <button className="px-6 py-3 bg-gradient-to-r from-yellow-400 via-
yellow-500 to-yellow-600 text-white font-semibold rounded-lg shadow-lg
hover:shadow-xl transition-all duration-300">
          <Link to="/collection">
            Explore Collection
          </Link>
        </button>
      </div>
    </div>
  </section>
  <PolicySection/>
</main>
)
}
export default About;

```

Contact.jsx :



```
import React from 'react'
import Title from '../components/Title'
import { assets } from '../assets/frontend_assets/assets'

const Contact = () => {
  return (
    <main className="pt-24 px-5 bg-gray-50 min-h-screen">
      <div className="text-center pt-12">
        <Title text1="CONTACT" text2="US" />
        <p className="mt-3 text-gray-500 max-w-xl mx-auto text-sm sm:text-base">
          Get in touch with us for any queries, orders, or support. We're here to help!
        </p>
      </div>

      <div className="my-16 flex flex-col md:flex-row items-center justify-center gap-12 max-w-6xl mx-auto">
        <div className="md:w-1/2 w-full">
          <img
            className="w-full rounded-2xl shadow-2xl hover:shadow-3xl transition-shadow duration-500 transform hover:scale-105"
            src={assets.contact_img}
            alt="Contact us"
          />
        </div>
      </div>
    </main>
  )
}
```

</div>

{/* Contact Details */}

<div className="md:w-1/2 w-full flex flex-col justify-center gap-6 text-gray-700 bg-white p-8 rounded-2xl shadow-lg">

<p className="text-xl font-semibold text-gray-800">Our Store</p>

<p className="text-gray-600 leading-relaxed">

 Woredrobe X Valsad Office

Shop No. 8, Shreeji Complex,

Near Tithal Road,

Valsad – 396001, Gujarat, India

</p>

<p className="text-gray-600 leading-relaxed">

 Tel: +91 8780800578

 Email: woredrobex@gmail.com

</p>

</div>

</section>

</main>

)

}

export default Contact;

Log in :

WARDROBE X

[Home](#) [Collection](#) [About](#) [Contact](#) [Log Out](#)

Log In

Welcome back to Men's Style

Email :

Password :

[Forgot-Password?](#)
Don't have an account? [Register](#)

Log In

```
import React, { useState } from "react";
import { Login as validateLogin } from "../validation";
import { login } from "../api/api";
import { useNavigate, Link } from "react-router-dom";

function Login() {
  const initialState = { email: "", password: "" };
  const [formData, setFormData] = useState(initialFormState);
  const [errors, setErrors] = useState({});
  const navigate = useNavigate();

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData((prev) => ({ ...prev, [name]: value }));
  };

  /* const handleSubmit = async (e) => {
    e.preventDefault();

    const { isValid, errors: validationErrors } = validateLogin(formData);
    setErrors(validationErrors);
  }; */
```

```

if (!isValid) return;

try {
  const response = await login(formData);
  alert(response.data.msg || "Login Successful!");

  // Save login info
  localStorage.setItem("token", response.data.token);
  localStorage.setItem("user", JSON.stringify(response.data.user));
  localStorage.setItem("isLoggedIn", "true");
  localStorage.setItem("userEmail", user.email || "");
  localStorage.setItem("userName", user.name || "");

  //  Notify Navbar
  window.dispatchEvent(new Event("storage"));

  setFormData(initialFormState);
  setErrors({});

  const role = response.data.user.role;
  if (role === "admin") {
    navigate("/Admin");
  } else {
    navigate("/");
  }
} catch (error) {
  if (error.response && error.response.data) {
    const { msg, error: apiError } = error.response.data;
    alert(msg || apiError || "Login failed");
  } else {
    alert("Something went wrong. Please try again.");
  }
}
};
*/

const handleSubmit = async (e) => {

```



```

e.preventDefault();

const { isValid, errors: validationErrors } = validateLogin(formData);
setErrors(validationErrors);

if (!isValid) return;

try {
  const response = await login(formData);
  alert(response.data.msg || "Login Successful!");

  const { user } = response.data; // ✅ fix here

  // ✅ Save login info
  localStorage.setItem("token", response.data.token);
  localStorage.setItem("user", JSON.stringify(user));
  localStorage.setItem("isLoggedIn", "true");
  localStorage.setItem("userEmail", user.email || "");
  localStorage.setItem("userName", user.name || "");

  // ✅ Notify Navbar about login status
  window.dispatchEvent(new Event("storage"));

  setFormData(initialFormState);
  setErrors({});

  // ✅ Redirect by role
  const role = user.role;
  if (role === "admin") {
    navigate("/Admin");
  } else {
    navigate("/");
  }
} catch (error) {
  if (error.response && error.response.data) {

```

```

const { msg, error: apiError } = error.response.data;
alert(msg || apiError || "Login failed");
} else {
  alert("Something went wrong. Please try again.");
}
}
};

```

```

return (
  <div className="min-h-screen w-full flex justify-center items-center font-
sans">

    <form
      onSubmit={handleSubmit}
      className="bg-white p-8 rounded-2xl shadow-lg w-full max-w-md border
border-gray-200"
    >
      <h1 className="text-3xl font-extrabold text-center mb-6 text-gray-900
tracking-wide">
        Log In
      </h1>
      <p className="text-center text-sm text-gray-500 mb-6">
        Welcome back to Men's Style
      </p>

      <div className="mb-5">
        <label htmlFor="email" className="block mb-1 font-medium text-gray-
700">
          Email :
        </label>
        <input
          type="email"
          name="email"
          id="email"
          value={formData.email}
          onChange={handleChange}

```

```

placeholder="example@gmail.com"
className="w-full px-4 py-3 border rounded-lg bg-gray-50 text-gray-900
placeholder-gray-400 focus:ring-2 focus:ring-yellow-500 focus:outline-none"
/>
{errors.email} && <p className="text-red-500 text-sm mt-
1">{errors.email}</p>
</div>

<div className="mb-6">
  <label htmlFor="password" className="block mb-1 font-medium text-gray-
700">
    Password :
  </label>
  <input
    type="password"
    name="password"
    id="password"
    value={formData.password}
    onChange={handleChange}
    placeholder="Enter Password"
    className="w-full px-4 py-3 border rounded-lg bg-gray-50 text-gray-900
placeholder-gray-400 focus:ring-2 focus:ring-yellow-500 focus:outline-none"
  />
  {errors.password} && <p className="text-red-500 text-sm mt-
1">{errors.password}</p>
</div>

<Link className="text-sm text-blue-600 hover:text-blue-800 hover:underline
transition" to="/forgot-Password">Forgot-Password?</Link>

<p className="text-sm mb-4 text-gray-500 text-center">
  Don't have an account? { " " }
  <Link to="/registration" className="text-yellow-600 font-medium
hover:underline">
    Register
  </Link>

```

</p>

<button

type="submit"

id="submit"

className="w-full bg-yellow-500 text-gray-900 py-3 rounded-lg hover:bg-yellow-400 transition-all shadow-md font-semibold tracking-wide"

>

Log In

</button>

</form>

</div>

);

}

export default Login;

Registration Page :

WARDROBE X

[Home](#) [Collection](#) [About](#) [Contact](#) [Log Out](#)

Sign Up

Join the Men's style community

Name :

Email :

Phone Number :

Password :

Already have an account? [Log In](#)

Register

```
// src/Pages/RegisForm/UserRegis.jsx

import React, { useState } from "react";

import { register } from "../api/api";

import { validateUserForm } from "../validation";

import { Link, useNavigate } from "react-router-dom";

function UserRegis() {

  const initialState = { name: "", email: "", phone: "", password: "" };

  const [formData, setFormData] = useState(initialFormState);

  const [errors, setErrors] = useState({});

  const navigate = useNavigate();

  const handleChange = (e) => {
```

```

const { name, value } = e.target;

setFormData((prev) => ({ ...prev, [name]: value }));

};

const handleSubmit = async (e) => {

  e.preventDefault();

  const { isValid, errors: validationErrors } = validateUserForm(formData);

  setErrors(validationErrors);

  if (!isValid) return;

  try {

    const response = await register(formData);

    alert(response.data.msg || "Registration successful!");

    setFormData(initialFormState);

    navigate("/login");

  } catch (error) {

    alert(error.response?.data?.error || "Already Registered");

  }

};

return (

  <main className="pt-24 px-5">

    <div className="flex justify-center items-center min-h-screen font-sans">

      <form

        onSubmit={handleSubmit}

```

```

        className="bg-white p-8 rounded-2xl shadow-lg w-full max-w-md border border-gray-200"
    >

    { /* Heading */ }

    <h1 className="text-3xl font-extrabold text-center mb-6 text-gray-900 tracking-wide">

        Sign Up

    </h1>

    <p className="text-center text-sm text-gray-500 mb-6">

        Join the Men's style community

    </p>

    { /* Name */ }

    <div className="mb-5">

        <label className="block mb-1 font-medium text-gray-700">Name :</label>

        <input

            type="text"

            name="name"

            value={formData.name}

            onChange={handleChange}

            placeholder="Enter Full Name"

            className="w-full px-4 py-3 border rounded-lg bg-gray-50 text-gray-900 focus:ring-2
focus:ring-yellow-500 focus:outline-none placeholder-gray-400"

        />

        {errors.name && (

            <p className="text-red-500 text-sm mt-1">{errors.name}</p>

        )}

    </div>

    { /* Email */ }

```

```

<div className="mb-5">

  <label className="block mb-1 font-medium text-gray-700">Email :</label>

  <input

    type="email"

    name="email"

    value={formData.email}

    onChange={handleChange}

    placeholder="example@gmail.com"

    className="w-full px-4 py-3 border rounded-lg bg-gray-50 text-gray-900 focus:ring-2
focus:ring-yellow-500 focus:outline-none placeholder-gray-400"

  />

  {errors.email && (

    <p className="text-red-500 text-sm mt-1">{errors.email}</p>

  )}

</div>

{/* Phone */}

<div className="mb-5">

  <label className="block mb-1 font-medium text-gray-700">

    Phone Number :

  </label>

  <input

    type="tel"

    name="phone"

    value={formData.phone}

    onChange={handleChange}

    placeholder="Enter 10 Digit Number"

```



```

        maxLength={10}

        className="w-full px-4 py-3 border rounded-lg bg-gray-50 text-gray-900 focus:ring-2
focus:ring-yellow-500 focus:outline-none placeholder-gray-400"

    />

    {errors.phone && (

        <p className="text-red-500 text-sm mt-1">{errors.phone}</p>

    )}

</div>

{/* Password */}

<div className="mb-6">

    <label className="block mb-1 font-medium text-gray-700">

        Password :

    </label>

    <input

        type="password"

        name="password"

        value={formData.password}

        onChange={handleChange}

        placeholder="Enter Password"

        className="w-full px-4 py-3 border rounded-lg bg-gray-50 text-gray-900 focus:ring-2
focus:ring-yellow-500 focus:outline-none placeholder-gray-400"

    />

    {errors.password && (

        <p className="text-red-500 text-sm mt-1">{errors.password}</p>

    )}

</div>

```

```

    { /* Login redirect */ }

    <p className="text-sm mb-4 text-gray-500 text-center">

        Already have an account? { " " }

        <Link

            to="/login"

            className="text-yellow-600 font-medium hover:underline"

        >

            Log In

        </Link>

    </p>

    { /* Submit button */ }

    <button

        type="submit"

        className="w-full bg-yellow-500 text-gray-900 py-3 rounded-lg hover:bg-yellow-400
transition-all shadow-md font-semibold tracking-wide"

    >

        Register

    </button>

</form>

</div>

</main>

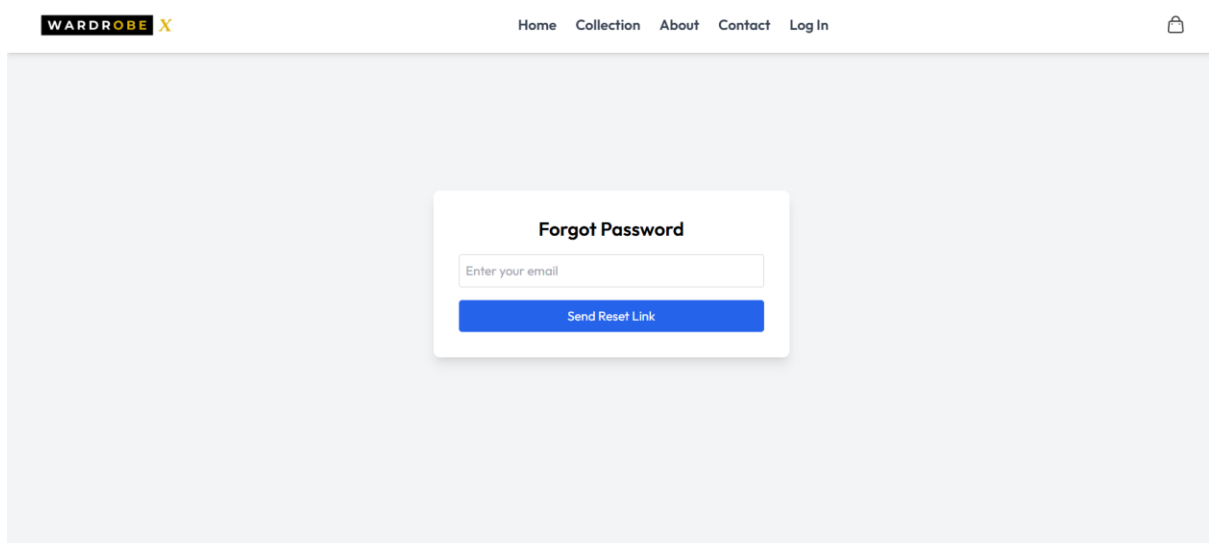
);

}

export default UserRegis;

```

ForgotPassword.jsx :



```
// src/pages/ForgotPassword.jsx
```

```
import React, { useState } from "react";
```

```
import axios from "axios";
```

```
import api from "../api/api";
```

```
const ForgotPassword = () => {
```

```
  const [email, setEmail] = useState("");
```

```
  const [message, setMessage] = useState("");
```

```
  const handleSubmit = async (e) => {
```

```
    e.preventDefault();
```

```
    try {
```

```
      const res = await api.post("/api/auth/forgot-password", { email });
```

```
      setMessage(res.data.message);
```

```
    } catch (err) {
```

```
      setMessage(err.response?.data?.message || "Something went wrong");
```

```
}  
};
```

```
return (
```

```
<div className="flex items-center justify-center h-screen bg-gray-100">
```

```
<form
```

```
  onSubmit={handleSubmit}
```

```
  className="bg-white shadow-lg rounded-lg p-8 w-full max-w-md"
```

```
>
```

```
<h2 className="text-2xl font-semibold mb-4 text-center">
```

```
  Forgot Password
```

```
</h2>
```

```
<input
```

```
  type="email"
```

```
  placeholder="Enter your email"
```

```
  value={email}
```

```
  onChange={(e) => setEmail(e.target.value)}
```

```
  required
```

```
  className="w-full p-2 border border-gray-300 rounded mb-4"
```

```
/>
```

```
<button
```

```
  type="submit"
```

```
  className="w-full bg-blue-600 text-white py-2 rounded hover:bg-blue-700"
```

```
>
```

```
  Send Reset Link
```

```
</button>
```

```
{message && <p className="mt-4 text-center text-gray-700">{message}</p>}
```

```
</form>
```

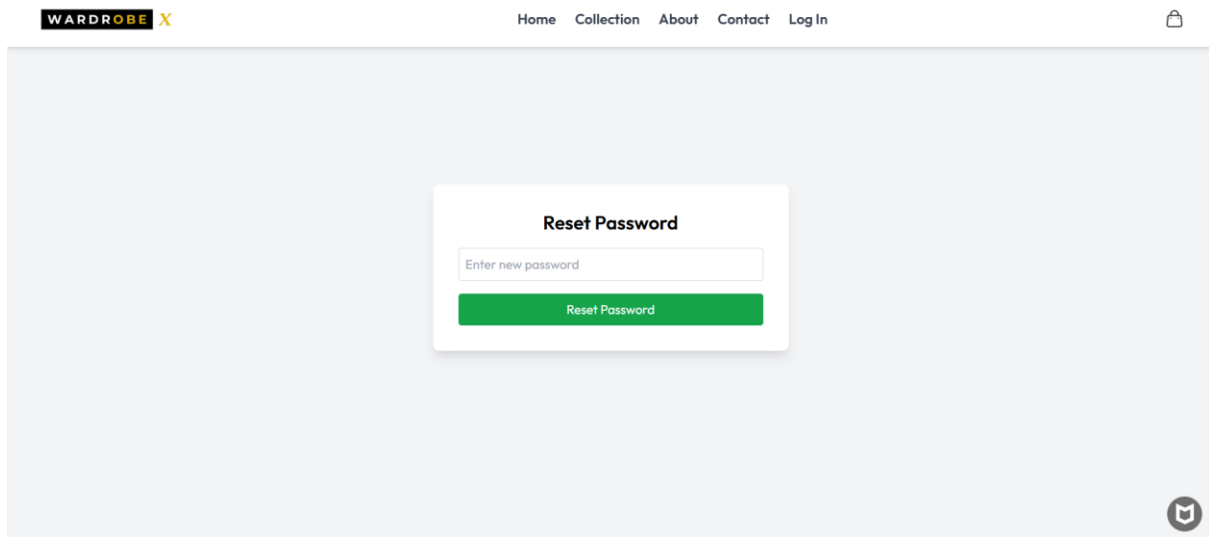
```
</div>
```

```
);
```

```
};
```

```
export default ForgotPassword;
```

ResetPassword :



```
// src/pages/ResetPassword.jsx
```

```
import React, { useState } from "react";
```

```
import { useParams, useNavigate } from "react-router-dom";
```

```
import api from "../api/api";
```

```
const ResetPassword = () => {
```

```
  const { token } = useParams();
```

```
  const navigate = useNavigate();
```

```
  const [newPassword, setNewPassword] = useState("");
```

```
  const [message, setMessage] = useState("");
```

```
  const handleReset = async (e) => {
```

```
    e.preventDefault();
```

```
    try {
```

```
      const res = await api.post(
```

```

    `/api/auth/reset-password/${token}`,

    { newPassword }

  );

  setMessage(res.data.message);

  setTimeout(() => navigate("/login"), 2000);

} catch (err) {

  setMessage(err.response?.data?.message || "Invalid or expired link");

}

};

```

```

return (

  <div className="flex items-center justify-center h-screen bg-gray-100">

    <form

      onSubmit={handleReset}

      className="bg-white shadow-lg rounded-lg p-8 w-full max-w-md"

    >

      <h2 className="text-2xl font-semibold mb-4 text-center">

        Reset Password

      </h2>

      <input

        type="password"

        placeholder="Enter new password"

        value={newPassword}

        onChange={(e) => setNewPassword(e.target.value)}

        required

        className="w-full p-2 border border-gray-300 rounded mb-4"

```

```
/>
```

```
<button
```

```
  type="submit"
```

```
  className="w-full bg-green-600 text-white py-2 rounded hover:bg-green-700"
```

```
>
```

```
  Reset Password
```

```
</button>
```

```
{message} && <p className="mt-4 text-center text-gray-700">{message}</p>}
```

```
</form>
```

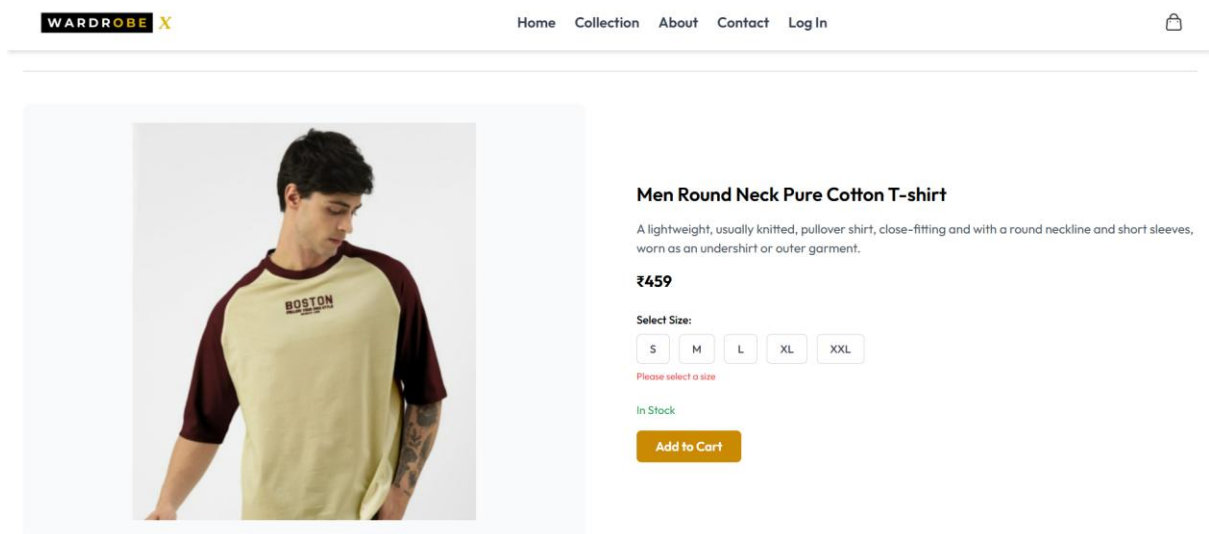
```
</div>
```

```
);
```

```
};
```

```
export default ResetPassword;
```


Product.jsx :



```
import React, { useEffect, useState, useContext } from "react";

import { useNavigate, useParams } from "react-router-dom";

import api from "../api/api"; // Axios instance

import { ShopContext } from "../context/ShopContext";

import { toast } from "react-toastify";

const Product = () => {

  const navigate = useNavigate();

  const { productId } = useParams();

  const [productData, setProductData] = useState(null);

  const [size, setSize] = useState("");

  const sizes = ["S", "M", "L", "XL", "XXL"];

  // ✅ Get addToCart from context

  const { addToCart: addToCartContext } = useContext(ShopContext);
```

```

// Fetch product details from backend

useEffect(() => {

  const fetchProduct = async () => {

    try {

      const res = await api.get(`/api/product/${productId}`);

      setProductData(res.data);

    } catch (err) {

      console.error("Error fetching product:", err);

      toast.error("Failed to fetch product data");

    }

  };

  fetchProduct();

}, [productId]);

if (!productData) return <div className="pt-24 text-center">Loading...</div>;

const getImageUrl = (imagePath) => `http://localhost:3000${imagePath}`;

//  Updated Add to Cart handler

const handleAddToCart = async () => {

  if (!size) {

    alert("Please select a size");

    return;

  }

```

```

const token = localStorage.getItem("token");

if (!token) {

  alert("You must login first!");

  navigate("/login"); // optional redirect

  return;

}

try {

  //  Use global context method (refreshes Navbar count)

  await addToCartContext(productData._id, size, 1);

  toast.success("Product added to cart!");

} catch (err) {

  if (err.response && err.response.status === 401) {

    alert("You must login first!");

    navigate("/login");

  } else {

    console.error("Error adding to cart:", err);

    toast.error("Failed to add product to cart");

  }

}

};

return (

  <main className="pt-24 px-5">

    <div className="border-t-2 pt-10">

```

```

<div className="flex flex-col sm:flex-row gap-12 sm:gap-16 items-center">

  {/* Product Image */}

  <div className="w-full sm:w-1/2 flex flex-col items-center justify-center bg-gray-50
rounded-xl p-6">

    <img

      src={productData.image}

      alt={productData.name}

      className="w-full max-h-[500px] object-contain rounded-lg"

    />

  </div>


  {/* Product Details */}

  <div className="w-full sm:w-1/2">

    <h2 className="text-2xl font-semibold mb-4">{productData.name}</h2>

    <p className="text-gray-600 mb-4">{productData.description}</p>

    <p className="text-xl font-bold mb-6">₹{productData.price}</p>


    {/* Size Selection */}

    <div className="mb-6">

      <h3 className="text-sm font-medium mb-2">Select Size:</h3>

      <div className="flex gap-3 flex-wrap mb-2">

        {sizes.map((s) => (

          <button

            key={s}

            onClick={() => setSize((prev) => (prev === s ? "" : s))}

            className={`px-4 py-2 border rounded-md text-sm font-medium transition-all ${

```

```

        size === s

        ? "bg-black text-white border-black"

        : "bg-white text-gray-700 border-gray-300 hover:border-black"

    }}

>

    {s}

</button>

)}}

</div>

{!size && (

    <p className="text-xs text-red-500 mt-1">

        Please select a size

    </p>

    )}

</div>

{/* Stock Info */}

<p

    className={`text-sm mb-4 ${

        productData.stock > 0

            ? "text-green-600"

            : "text-red-500 font-medium"

        }}

>

    {productData.stock > 0 ? "In Stock" : "Out of Stock"}

</p>

```

```

    { /* Add to Cart Button */}

    <button

      onClick={handleAddToCart}

      disabled={productData.stock === 0}

      className={`px-6 py-2 rounded-md text-white font-semibold transition ${
        productData.stock === 0
          ? "bg-gray-400 cursor-not-allowed"
          : "bg-yellow-600 hover:bg-yellow-700"
        }`}

    >

      Add to Cart

    </button>

  </div>

</div>

</div>

</main>

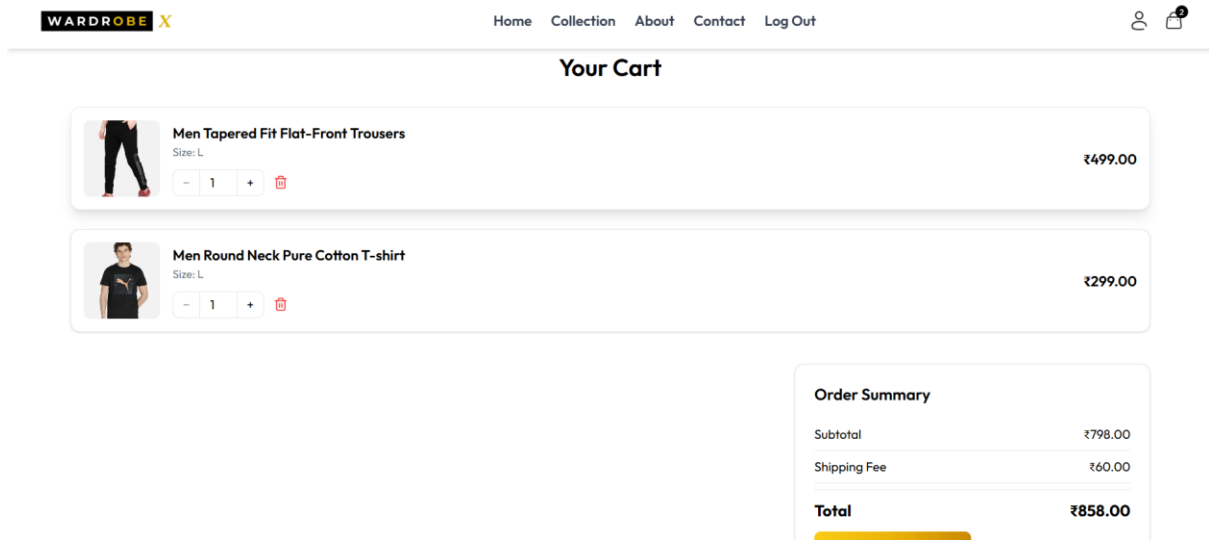
);

};

export default Product;

```

Cart.jsx :



```
import React, { useEffect, useState } from "react";  
  
import { useNavigate } from "react-router-dom";  
  
import { Trash2 } from "lucide-react";  
  
import api from "../api/api";
```

```
const Cart = () => {  
  
  const [cartList, setCartList] = useState([]);  
  
  const [currency] = useState("₹");  
  
  const [deliveryFee] = useState(60);  
  
  const navigate = useNavigate();  
  
  const fetchCart = async () => {  
  
    try {  
  
      const token = localStorage.getItem("token");  
  
      const res = await api.get("/api/cart", {
```

```
headers: { Authorization: `Bearer ${token}` },  
});
```

```
// 🔥 Filter out invalid/null product items
```

```
const items = res.data.items  
  
  .filter((item) => item.product !== null)  
  
  .map((item) => ({  
  
    product: item.product,  
  
    size: item.size,  
  
    qty: item.quantity,  
  
  }));
```

```
setCartList(items);
```

```
} catch (err) {  
  
  console.error("Error fetching cart:", err);  
  
}  
};
```

```
useEffect(() => {  
  
  fetchCart();  
  
}, []);
```

```
// 🔥 Safe subtotal calculation
```

```
const subtotal = cartList.reduce((acc, item) => {  
  
  if (!item.product || item.product.price === null) return acc;
```



```
    return acc + item.product.price * item.qty;
  }, 0);
```

```
const shippingFee = subtotal > 0 ? deliveryFee : 0;

const total = subtotal + shippingFee;
```

```
const handleCheckout = () => {

  const isLoggedIn = localStorage.getItem("isLoggedIn") === "true";

  if (!isLoggedIn) {

    alert("You must log in first!");

    navigate("/login");

  } else {

    navigate("/placeorder");

  }

};
```

```
const handleRemove = async (productId, size) => {

  try {

    const token = localStorage.getItem("token");

    await api.delete("/api/cart/remove", {

      headers: { Authorization: `Bearer ${token}` },

      data: { productId, size },

    });

    fetchCart();

  } catch (err) {

    console.error(err);

  }

};
```

```

    }
};

const handleUpdateQuantity = async (productId, size, quantity) => {
  if (quantity < 1) return;

  try {
    const token = localStorage.getItem("token");

    await api.put(
      "/api/cart/update",
      { productId, size, quantity },
      { headers: { Authorization: `Bearer ${token}` } }
    );

    fetchCart();
  } catch (err) {
    console.error(err);
  }
};

if (cartList.length === 0) {
  return (
    <main className="pt-24 px-5 text-center">
      <h2 className="text-3xl font-semibold mb-4">Your Cart</h2>
      <p className="text-gray-600">Your cart is empty.</p>
    </main>
  );
}

```

```
return (
```

```
<main className="pt-24 px-5 md:px-20 pb-10">
```

```
<h2 className="text-3xl font-semibold mb-8 text-center">Your Cart</h2>
```

```
<div className="flex flex-col gap-6">
```

```
{cartList
```

```
.filter((item) => item.product) // 🔥 Skip null products in UI
```

```
.map(({ product, size, qty }) => (
```

```
<div
```

```
key={` ${product._id}-${size} `}
```

```
className="flex flex-col md:flex-row justify-between items-center border p-4 rounded-xl  
shadow bg-white hover:shadow-lg transition"
```

```
>
```

```
<div className="flex items-center gap-4 w-full md:w-2/3">
```

```
<img
```

```
src={` http://localhost:3000${product.image} `}
```

```
alt={product.name}
```

```
className="w-24 h-24 object-cover rounded-lg"
```

```
/>
```

```
<div className="flex flex-col flex-1">
```

```
<h3 className="font-semibold text-lg">{product.name}</h3>
```

```
<p className="text-sm text-gray-500">Size: {size}</p>
```

```
<div className="flex items-center gap-3 mt-3">
```

```
<div className="flex items-center border rounded-lg overflow-hidden">
```

```
<button
```

```
  onClick={() =>
```

```
    handleUpdateQuantity(product._id, size, qty - 1)
```

```
  }
```

```
  disabled={qty === 1}
```

```
  className={`px-3 py-1 ${
```

```
    qty === 1
```

```
      ? "text-gray-400 cursor-not-allowed"
```

```
      : "hover:bg-gray-100"
```

```
    }`}
```

```
>
```

```
  —
```

```
</button>
```

```
<input
```

```
  type="number"
```

```
  min="1"
```

```
  value={qty}
```

```
  onChange={(e) =>
```

```
    handleUpdateQuantity(
```

```
      product._id,
```

```
      size,
```

```
      Number(e.target.value)
```

```
    )
```

```
  }
```

```
  className="w-12 text-center border-l border-r py-1"
```

```
/>
```

```
<button
```

```
  onClick={() =>
```

```
    handleUpdateQuantity(product._id, size, qty + 1)
```

```
  }
```

```
  className="px-3 py-1 hover:bg-gray-100"
```

```
>
```

```
  +
```

```
</button>
```

```
</div>
```

```
<button
```

```
  onClick={() => handleRemove(product._id, size)}
```

```
  className="text-red-500 hover:text-red-700"
```

```
  title="Remove product"
```

```
>
```

```
  <Trash2 size={18} />
```

```
</button>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div className="mt-4 md:mt-0 font-semibold text-lg">
```

```
  {currency}
```

```
  {(product.price * qty).toFixed(2)}
```

```
</div>
```

</div>

)))

</div>

<div className="mt-10 max-w-md ml-auto p-6 bg-white border rounded-xl shadow">

<h3 className="text-xl font-semibold mb-4">Order Summary</h3>

<div className="flex justify-between py-2 border-b">

Subtotal

{currency}

{subtotal.toFixed(2)}

</div>

<div className="flex justify-between py-2 border-b">

Shipping Fee

{currency}

{shippingFee.toFixed(2)}

</div>

<div className="flex justify-between py-3 text-xl font-bold border-t mt-2">

Total


```
        {currency}

        {total.toFixed(2)}

    </span>

</div>


<button

    onClick={handleCheckout}

    className="px-6 py-3 bg-gradient-to-r from-yellow-400 via-yellow-500 to-yellow-600 text-
white font-semibold rounded-lg shadow-lg hover:shadow-xl transition-all duration-300"

    >

        Proceed to Checkout

    </button>

</div>

</main>

);

};

export default Cart;
```

PlaceOrder.jsx :

WARDROBE X

HomeCollectionAboutContactLog Out



Checkout

Billing Details

Yash Patel

yashpatel7415@gmail.com

Chikhla, Lilapore, Valsad

Valsad

Gujarat

396375

856321475d

Payment Method

☒ Cash on Delivery

☐ Card Payment

☐ UPI

Order Summary

Men Tapered Fit Flat-Front Trousers (Size: L) x 1	₹499.00
Men Round Neck Pure Cotton T-shirt (Size: L) x 1	₹299.00
Subtotal	₹798.00
Shipping Fee	₹60.00
Total	₹858.00

Place Order

```
import React, { useContext, useState, useEffect } from "react";

import { ShopContext } from "../context/ShopContext";

import { validateCheckoutForm } from "../validation";

import api from "../api/api";

import { Navigate } from "react-router-dom";

const Checkout = () => {

  const { currency, delivery_fee, cartItems, products, fetchCart } = useContext(ShopContext);

  const [formData, setFormData] = useState({

    name: "",

    email: "",

    address: "",

    city: "",

    state: "",

    zip: "",
```



```
    phone: "",
    paymentMethod: "cod",
    cardName: "",
    cardNumber: "",
    expiry: "",
    cvv: "",
    upiId: "",
  });
```

```
const [errors, setErrors] = useState({});
const [cartList, setCartList] = useState([]);
const [loading, setLoading] = useState(true);
```

```
//  Auto-fill name and email from logged-in user (localStorage)
```

```
useEffect(() => {
  const user = JSON.parse(localStorage.getItem("user"));
  if (user) {
    setFormData((prev) => ({
      ...prev,
      name: user.name || "",
      email: user.email || "",
    }));
  }
}, []);
```

```

// Build cart list from cartItems & products

useEffect(() => {

  const list = [];

  for (const productId in cartItems) {

    for (const size in cartItems[productId]) {

      const qty = cartItems[productId][size] || 0;

      const product = products.find((p) => p._id === productId);

      if (!product || qty <= 0) continue;

      list.push({ product, size, qty });

    }

  }

  setCartList(list);

  setLoading(false);

}, [cartItems, products]);

const subtotal = cartList.reduce((acc, item) => acc + item.product.price * item.qty, 0);

const shippingFee = subtotal > 0 ? delivery_fee : 0;

const totalAmount = subtotal + shippingFee;

const handleChange = (e) => {

  const { name, value } = e.target;

  setFormData((prev) => ({ ...prev, [name]: value }));

};

const handleSubmit = async (e) => {

  e.preventDefault();

```

```
const { isValid, errors } = validateCheckoutForm(formData);
```

```
setErrors(errors);
```

```
if (!isValid) return;
```

```
if (cartList.length === 0) {
```

```
  alert("🛒 Your cart is empty!");
```

```
  return;
```

```
}
```

```
try {
```

```
  const token = localStorage.getItem("token");
```

```
  if (!token) throw new Error("User not logged in!");
```

```
const orderData = {
```

```
  customerName: formData.name,
```

```
  email: formData.email,
```

```
  phone: formData.phone,
```

```
  address: formData.address,
```

```
  city: formData.city,
```

```
  state: formData.state,
```

```
  zip: formData.zip,
```

```
  paymentMethod: formData.paymentMethod,
```

```
  totalAmount,
```

```
  items: cartList.map((item) => ({
```

```

        productId: item.product._id,

        name: item.product.name,

        price: item.product.price,

        qty: item.qty,

        size: item.size,

    })),

};

const res = await api.post("/api/orders", orderData, {

    headers: { Authorization: `Bearer ${token}` },

});

if (res.data.success) {

    alert("✅ Order placed successfully!");

    setFormData((prev) => ({

        ...prev,

        address: "",

        city: "",

        state: "",

        zip: "",

        phone: "",

        paymentMethod: "cod",

        cardName: "",

        cardNumber: "",

        expiry: "",

```

```

        cvv: "",
        upiId: "",
    }));
    fetchCart();

;

}

} catch (err) {

    console.error(err.response || err);

    alert(err.response?.data?.message || err.message || " ❌ Failed to place order");

}

};

if (loading)

    return (

        <main className="pt-24 px-5 text-center">

            <h2 className="text-2xl font-semibold mb-4">Checkout</h2>

            <p>Loading your cart...</p>

        </main>

    );

if (cartList.length === 0)

    return (

        <main className="pt-24 px-5 text-center">

```

```
<h2 className="text-2xl font-semibold mb-4">Checkout</h2>
```

```
<p>Your cart is empty.</p>
```

```
</main>
```

```
);
```

```
const inputClass = (field) =>
```

```
`w-full border rounded px-3 py-2 ${
```

```
errors[field] ? "border-red-500" : "border-gray-300"
```

```
} focus:outline-none focus:ring-2 focus:ring-blue-400`;
```

```
return (
```

```
<main className="pt-24 px-5 pb-16">
```

```
<h2 className="text-2xl font-semibold mb-6 text-center">Checkout</h2>
```

```
<form onSubmit={handleSubmit} className="max-w-5xl mx-auto grid md:grid-cols-2 gap-8">
```

```
  { /* LEFT: Billing Details */ }
```

```
<div className="space-y-4 border p-6 rounded-xl shadow-md bg-white">
```

```
<h3 className="text-lg font-medium mb-4">Billing Details</h3>
```

```
{["name", "email", "address", "city", "state", "zip", "phone"].map((field) => (
```

```
<div key={field}>
```

```
<input
```

```
  name={field}
```

```
  placeholder={field.charAt(0).toUpperCase() + field.slice(1)}
```

```
  value={formData[field]}
```

```
  onChange={handleChange}
```

```

        readOnly={field === "email"} // 🖱️ make email read-only

        className={` ${inputClass(field)} ${
            field === "email" ? "bg-gray-100 cursor-not-allowed" : ""
        }}

    />

    {errors[field] && <p className="text-red-500 text-sm">{errors[field]}</p>}

</div>

)}}

</div>

```

```

{/* RIGHT: Payment & Summary */}

<div className="space-y-6">

    {/* Payment Method */}

    <div className="border p-6 rounded-xl shadow-md bg-white space-y-4">

        <h3 className="text-lg font-medium mb-2">Payment Method</h3>

        {[ "cod", "card", "upi" ].map((method) => (

            <label key={method} className="flex items-center gap-2">

                <input

                    type="radio"

                    name="paymentMethod"

                    value={method}

                    checked={formData.paymentMethod === method}

                    onChange={handleChange}

                />

                {method === "cod"

```

```

        ? "Cash on Delivery"

        : method === "card"

        ? "Card Payment"

        : "UPI"}

</label>

)}}

```

```

{formData.paymentMethod === "card" && (

  <div className="space-y-2 mt-2">

    {[ "cardName", "cardNumber", "expiry", "cvv"].map((field) => (

      <input

        key={field}

        name={field}

        placeholder={field}

        value={formData[field]}

        onChange={handleChange}

        className={inputClass(field)}

      />

    ))}

  </div>

)}

```

```

{formData.paymentMethod === "upi" && (

  <div>

    <input

      name="upiId"

```



```

        placeholder="UPI ID"

        value={formData.upiId}

        onChange={handleChange}

        className={inputClass("upiId")}

    />

</div>

)}}

</div>

{/* Order Summary */}

<div className="border p-6 rounded-xl shadow-md bg-white space-y-4">

    <h3 className="text-lg font-medium mb-2">Order Summary</h3>

    {cartList.map((item) => (

        <div key={item.product._id + item.size} className="flex justify-between text-sm">

            <span>

                {item.product.name} (Size: {item.size}) x {item.qty}

            </span>

            <span>

                {currency}

                {(item.product.price * item.qty).toFixed(2)}

            </span>

        </div>

    ))}

    <div className="flex justify-between border-t pt-2 mt-2">

        <span>Subtotal</span>

```

```
<span>

    {currency}

    {subtotal.toFixed(2)}

</span>

</div>

<div className="flex justify-between">

    <span>Shipping Fee</span>

    <span>

        {currency}

        {shippingFee.toFixed(2)}

    </span>

</div>

<div className="flex justify-between font-bold text-lg border-t pt-2 mt-2">

    <span>Total</span>

    <span>

        {currency}

        {totalAmount.toFixed(2)}

    </span>

</div>

<button

    type="submit"

    className="px-32 py-3 bg-gradient-to-r from-yellow-400 via-yellow-500 to-yellow-600
text-white font-semibold rounded-lg shadow-lg hover:shadow-xl transition-all duration-300"

    >

    Place Order
```

```
        </button>

    </div>

</div>

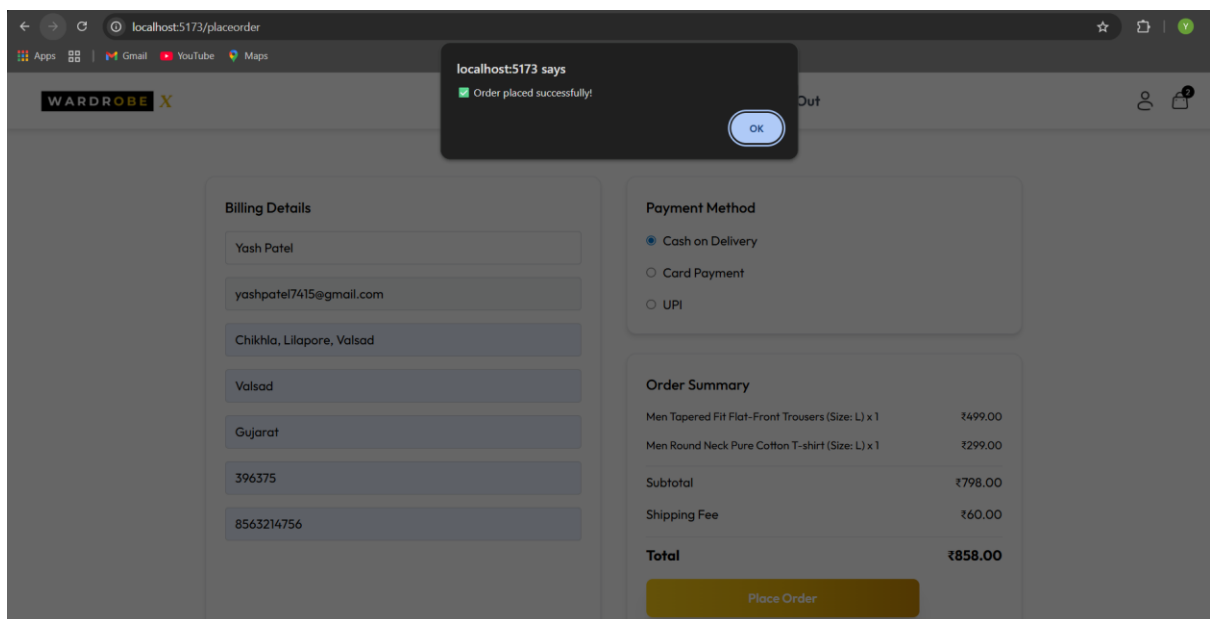
</form>

</main>

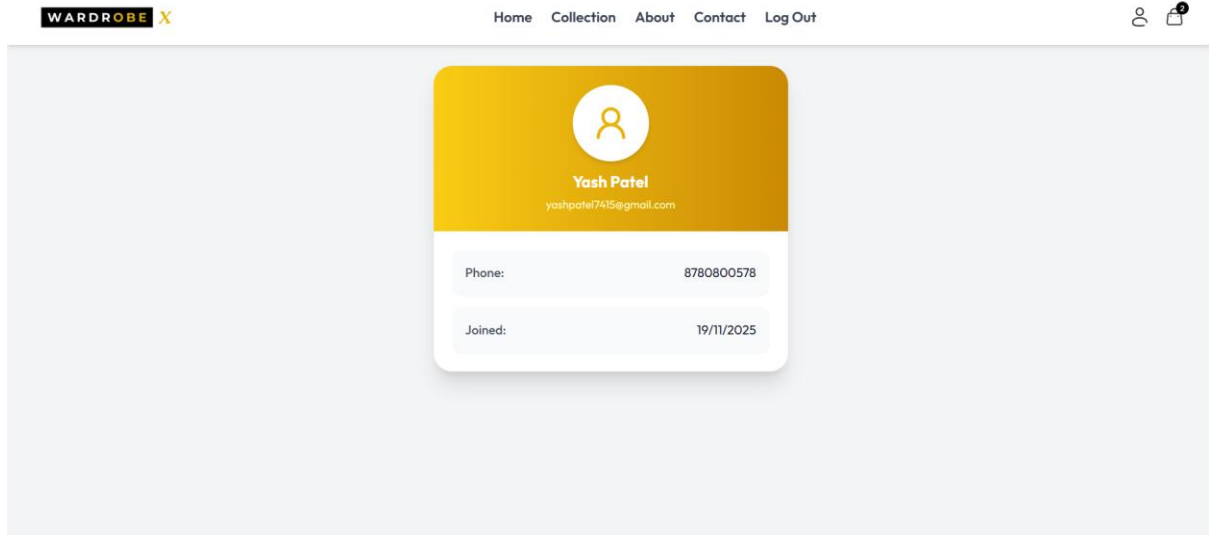
);

};

export default Checkout;
```



MyProfile.jsx :



```
// src/Pages/Profile/MyProfile.jsx
```

```
import React, { useEffect, useState } from "react";
```

```
import { User2 } from "lucide-react"; // optional icon for avatar
```

```
const MyProfile = () => {
```

```
  const [user, setUser] = useState(null);
```

```
  useEffect(() => {
```

```
    const storedUser = localStorage.getItem("user");
```

```
    if (storedUser) {
```

```
      setUser(JSON.parse(storedUser));
```

```
    }
```

```
  }, []);
```

```
  if (!user) {
```

```

return (

  <div className="flex justify-center items-center min-h-screen text-gray-500 text-lg font-
medium">

    No user data found. Please log in.

  </div>

);

}

return (

  <div className="min-h-screen flex flex-col items-center pt-24 px-5 bg-gray-100">

    <div className="bg-white rounded-3xl shadow-xl w-full max-w-md border border-gray-200
overflow-hidden">

      { /* Header with gradient */ }

      <div className="bg-gradient-to-r from-yellow-400 to-yellow-600 p-6 flex flex-col items-
center">

        <div className="w-24 h-24 bg-white rounded-full flex items-center justify-center shadow-
md mb-3">

          <User2 className="w-12 h-12 text-yellow-500" />

        </div>

        <h1 className="text-white text-xl font-bold">{user.name}</h1>

        <p className="text-yellow-100 mt-1 text-sm">{user.email}</p>

      </div>

      { /* Body */ }

      <div className="p-6 space-y-4">

        {user.phone && (

          <div className="flex justify-between items-center bg-gray-50 p-4 rounded-xl shadow-sm
hover:shadow-md transition">

```

```

        <span className="text-gray-600 font-medium">Phone:</span>

        <span className="text-gray-900">{user.phone}</span>

    </div>

)}

{user.isAdmin && (

    <div className="flex justify-between items-center bg-gray-50 p-4 rounded-xl shadow-sm
hover:shadow-md transition">

        <span className="text-gray-600 font-medium">Role:</span>

        <span className="text-gray-900">Admin</span>

    </div>

)}

{/* Placeholder for future info */}

<div className="flex justify-between items-center bg-gray-50 p-4 rounded-xl shadow-sm
hover:shadow-md transition">

    <span className="text-gray-600 font-medium">Joined:</span>

    <span className="text-gray-900">

        {new Date(user.createdAt || Date.now()).toLocaleDateString()}

    </span>

</div>

</div>

</div>

</div>

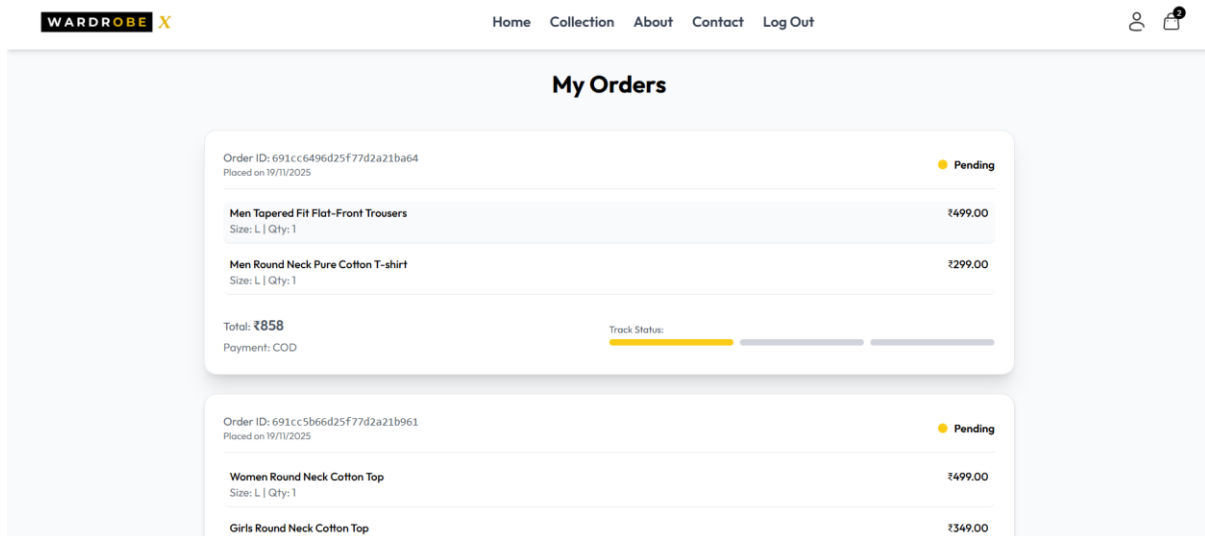
);

};

export default MyProfile;

```

MyOrders.jsx :



```
import React, { useEffect, useState } from "react";
```

```
import api from "../api/api";
```

```
const MyOrders = () => {
```

```
  const [orders, setOrders] = useState([]);
```

```
  const [loading, setLoading] = useState(true);
```

```
  const [error, setError] = useState("");
```

```
  const statusColors = {
```

```
    Pending: "bg-yellow-400",
```

```
    Shipped: "bg-blue-400",
```

```
    Delivered: "bg-green-500",
```

```
    Cancelled: "bg-red-500",
```

```
  };
```

```

useEffect(() => {

  const fetchOrders = async () => {

    try {

      const token = localStorage.getItem("token");

      if (!token) throw new Error("Not logged in");

      const res = await api.get("/api/orders", {

        headers: { Authorization: `Bearer ${token}` },

      });

      setOrders(res.data.orders || []);

    } catch (err) {

      console.error("Error fetching orders:", err);

      setError(err.response?.data?.message || err.message);

    } finally {

      setLoading(false);

    }

  };

```

```

    fetchOrders();

  }, []);

```

```

if (loading) {

```

```

  return (

```

```

    <main className="pt-24 text-center min-h-screen flex flex-col items-center justify-center bg-
    gray-50">

```



```

    <h2 className="text-2xl font-semibold mb-2">My Orders</h2>

    <p className="text-gray-500">Loading your orders...</p>

  </main>

);

}

if (error) {

  return (

    <main className="pt-24 text-center min-h-screen flex flex-col items-center justify-center bg-gray-50">

      <h2 className="text-2xl font-semibold mb-2">My Orders</h2>

      <p className="text-red-600">{error}</p>

    </main>

  );

}

if (orders.length === 0) {

  return (

    <main className="pt-24 text-center min-h-screen flex flex-col items-center justify-center bg-gray-50">

      <h2 className="text-2xl font-semibold mb-2">My Orders</h2>

      <p className="text-gray-500">You haven't placed any orders yet.</p>

    </main>

  );

}

```

```

return (

  <main className="pt-24 pb-16 px-4 sm:px-10 bg-gray-50 min-h-screen">

    <h2 className="text-3xl font-bold mb-10 text-center">My Orders</h2>

    <div className="space-y-6 max-w-5xl mx-auto">

      {orders.map((order) => (

        <div

          key={order._id}

          className="border rounded-2xl p-6 bg-white shadow-md hover:shadow-xl transition-all
duration-300"

          >

            { /* Header */ }

            <div className="flex justify-between items-center flex-wrap gap-3 border-b pb-3 mb-4">

              <div>

                <p className="text-gray-600 text-sm">

                  Order ID: <span className="font-mono">{order._id}</span>

                </p>

                <p className="text-gray-500 text-xs">

                  Placed on {new Date(order.createdAt).toLocaleDateString()}

                </p>

              </div>

            <div className="flex items-center gap-2">

              <span

                className={`w-3 h-3 rounded-full ${

                  statusColors[order.status] || "bg-gray-400"

                }`}

              >

```

```

    ></span>

    <span className="font-semibold text-sm">{order.status}</span>

  </div>

</div>

{/* Items */}

<div className="space-y-3">

  {order.items.map((item, index) => (

    <div

      key={index}

      className="flex justify-between text-sm border-b pb-2 hover:bg-gray-50 transition
rounded-lg px-2 py-1"

    >

      <div>

        <p className="font-medium">{item.name}</p>

        <p className="text-gray-500 text-xs sm:text-sm">

          Size: {item.size} | Qty: {item.qty}

        </p>

      </div>

      <p className="font-medium">

        ₹{(item.price * item.qty).toFixed(2)}

      </p>

    </div>

    )))

</div>

```

```
{/* Order Summary & Tracking */}
```

```
<div className="flex flex-col sm:flex-row justify-between items-start sm:items-center mt-6 gap-4 sm:gap-0">
```

```
<div className="space-y-1">
```

```
<p className="text-gray-600 text-sm">
```

```
Total: <span className="font-semibold text-lg">₹{order.totalAmount}</span>
```

```
</p>
```

```
<p className="text-gray-500 text-xs sm:text-sm">
```

```
Payment: {order.paymentMethod?.toUpperCase()}
```

```
</p>
```

```
</div>
```

```
{/* Tracking bar */}
```

```
<div className="w-full sm:w-1/2">
```

```
<p className="text-gray-600 text-xs mb-1">Track Status:</p>
```

```
<div className="flex items-center gap-2 w-full h-2">
```

```
{["Pending", "Shipped", "Delivered"].map((step, idx) => {
```

```
const active =
```

```
["Pending", "Shipped", "Delivered"].indexOf(order.status) >= idx;
```

```
return (
```

```
<div
```

```
key={step}
```

```
className={`flex-1 h-2 rounded-full transition-colors duration-300 ${
```

```
active ? statusColors[order.status] : "bg-gray-300"
```

```
}`}
```

```
></div>
```

```

        );

    }
}

</div>

</div>

</div>

</div>

)}}

</div>

</main>

);

};

```

```
export default MyOrders;
```

Api :

```

import axios from "axios";

const api = axios.create({

  baseURL: "http://localhost:3000", // your backend base URL

});

// 🗝 Automatically attach token to every request if available

api.interceptors.request.use(

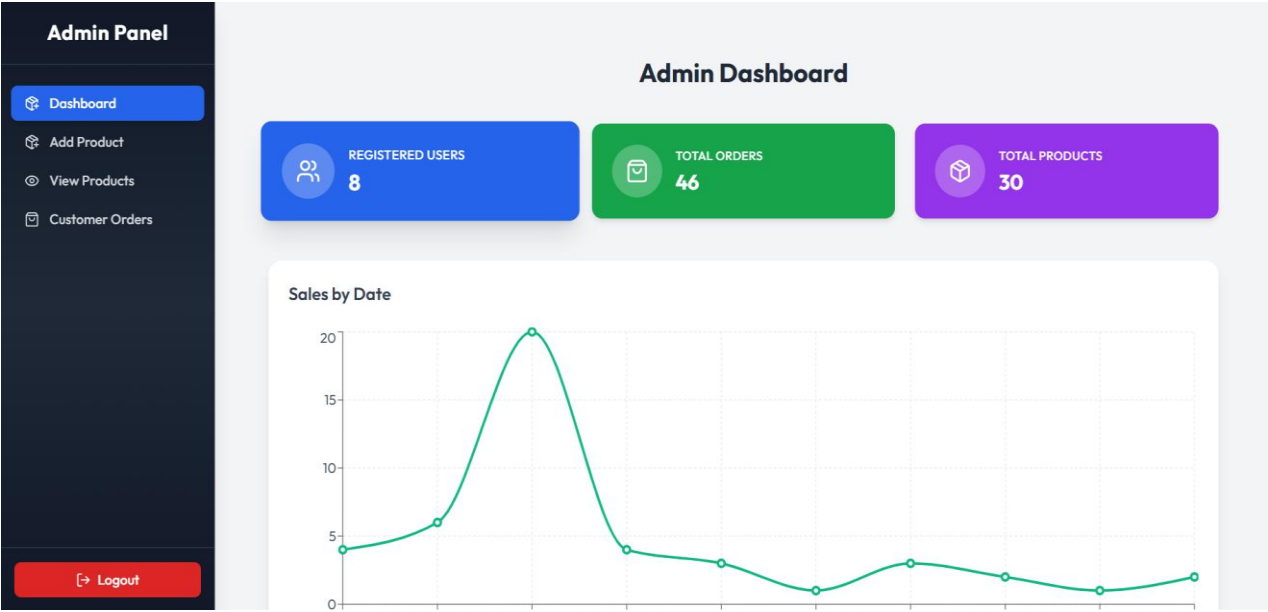
  (config) => {

    const token = localStorage.getItem("token");

```

```
    if (token) {  
      config.headers.Authorization = `Bearer ${token}`;  
    }  
    return config;  
  },  
  (error) => Promise.reject(error)  
);  
  
// 🌱 API Endpoints  
  
export const register = (formData) => api.post("/add", formData);  
  
export const login = (formData) => api.post("/login", formData);  
  
export default api;
```

Admin Side :



AdminSideBar :

```
import React from "react";
import { NavLink, useNavigate } from "react-router-dom";
import { Logout, PackagePlus, Eye, ShoppingBag } from "lucide-react";
```

```

const AdminSidebar = () => {
  const navigate = useNavigate();

  const handleLogout = () => {
    localStorage.removeItem("adminToken");
    localStorage.setItem("isLoggedIn", "false");
    window.dispatchEvent(new Event("storage"));
    navigate("/");
  };

  const menuItems = [
    { name: "Dashboard", path: "/admin", icon: <PackagePlus size={18} /> },
    { name: "Add Product", path: "/admin/add-product", icon: <PackagePlus size={18} /> },
    { name: "View Products", path: "/admin/view-products", icon: <Eye size={18} /> },
    { name: "Customer Orders", path: "/admin/view-orders", icon: <ShoppingBag size={18} /> },
  ];

  return (
    <aside className="w-64 h-screen bg-gradient-to-b from-gray-900 via-gray-800 to-gray-900
text-gray-200 flex flex-col shadow-xl sticky top-0 border-r border-gray-700">
      </* Header */>
      <div className="p-5 border-b border-gray-700">
        <h1 className="text-2xl font-bold text-center text-white tracking-wide">
          Admin Panel
        </h1>
      </div>

      </* Navigation */>
      <nav className="flex-1 mt-6 flex flex-col gap-1 px-3">
        {menuItems.map((item) => (
          <NavLink
            key={item.name}
            to={item.path}
            className={({ isActive }) =>

```



```

        `flex items-center gap-3 px-4 py-2 rounded-lg font-medium transition-all duration-200
    ${
        isActive
        ? "bg-blue-600 text-white shadow-md"
        : "hover:bg-gray-700 hover:text-white text-gray-300"
    }`
    }
    >
    {item.icon}
    <span>{item.name}</span>
    </NavLink>
  )})
</nav>

{/* Logout Button */}
<div className="p-4 border-t border-gray-700">
  <button
    onClick={handleLogout}
    className="w-full flex items-center justify-center gap-2 bg-red-600 hover:bg-red-700
text-white font-medium py-2 rounded-lg transition-all duration-200"
  >
    <LogOut size={18} />
    Logout
  </button>
</div>
</aside>

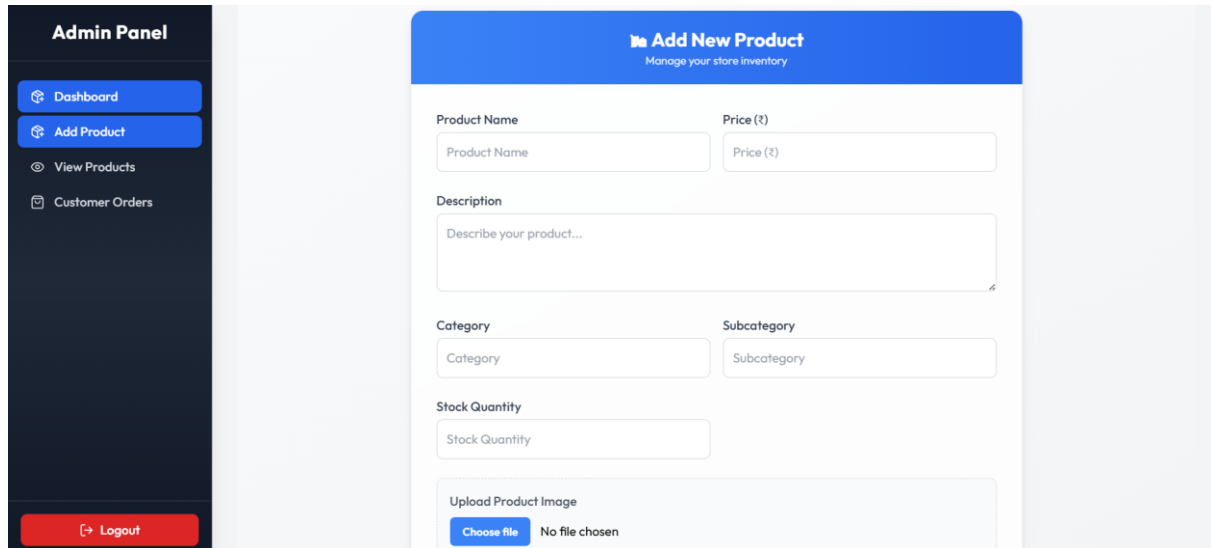
);
};

export default AdminSidebar;

```

Admin Pages :

AddProduct.jsx



The screenshot displays the 'Admin Panel' on the left and the 'Add New Product' form on the right. The Admin Panel sidebar includes links for Dashboard, Add Product, View Products, and Customer Orders, along with a Logout button. The 'Add New Product' form is titled 'Manage your store inventory' and contains the following fields:

- Product Name (text input)
- Price (₹) (text input)
- Description (text area with placeholder 'Describe your product...')
- Category (text input)
- Subcategory (text input)
- Stock Quantity (text input)
- Upload Product Image (file upload button labeled 'Choose file' and 'No file chosen')

```
import React, { useState } from 'react';
import api from '../api/api';
import { validateProductForm } from '../validation';
```

```
const AddProduct = () => {
  const [product, setProduct] = useState({
    name: "",
    description: "",
    price: "",
    category: "",
    subCategory: "",
    size: "",
    stock: "",
    image: null
  });
```

```
const [errors, setErrors] = useState({});
```

```

const [imagePreview, setImagePreview] = useState(null);

const handleChange = (e) => {
  setProduct({ ...product, [e.target.name]: e.target.value });
  setErrors({ ...errors, [e.target.name]: " " });
};

const handleImageChange = (e) => {
  const file = e.target.files[0];
  if (file) {
    setProduct({ ...product, image: file });
    setImagePreview(URL.createObjectURL(file));
    setErrors({ ...errors, image: " " });
  }
};

const handleSubmit = async (e) => {
  e.preventDefault();

  const { isValid, errors: validationErrors } = validateProductForm(product);
  if (!isValid) {
    setErrors(validationErrors);
    return;
  }

  try {
    const formData = new FormData();
    formData.append('name', product.name);
    formData.append('description', product.description);
    formData.append('price', product.price);
    formData.append('category', product.category);
    formData.append('subCategory', product.subCategory);
    formData.append('stock', product.stock);
    //formData.append('size', JSON.stringify(product.size.split(',').map(s => s.trim())));

    if (product.image) formData.append('image', product.image);
  }
};

```

```
await api.post('/api/product', formData, {
  headers: { 'Content-Type': 'multipart/form-data' },
});
```

```
alert('✅ Product added successfully!');
```

```
setProduct({
  name: "",
  description: "",
  price: "",
  category: "",
  subCategory: "",
  //size: "",
  stock: "",
  image: null
});
setErrors({});
setImagePreview(null);
} catch (err) {
  console.error(err.response || err);
  alert(err.response?.data?.message || '❌ Error adding product');
}
};
```

```
return (
  <div className="min-h-screen flex items-center justify-center bg-gradient-to-br from-gray-50 to-gray-100 p-8">
    <div className="w-full max-w-3xl bg-white/80 backdrop-blur-md border border-gray-200 shadow-lg rounded-2xl overflow-hidden">
      { /* Header */ }
      <div className="bg-gradient-to-r from-blue-500 to-blue-600 text-white text-center py-5">
        <h2 className="text-2xl font-bold">🛒 Add New Product</h2>
        <p className="text-sm opacity-90">Manage your store inventory</p>
      </div>
```

```

    { /* Form */ }
    <form onSubmit={handleSubmit} className="p-8 space-y-6">
      { /* Name & Price */ }
      <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
        <InputField
          label="Product Name"
          name="name"
          value={product.name}
          onChange={handleChange}
          error={errors.name}
        />
        <InputField
          label="Price (₹)"
          name="price"
          type="number"
          value={product.price}
          onChange={handleChange}
          error={errors.price}
        />
      </div>

      { /* Description */ }
      <div>
        <label className="block text-gray-700 font-medium mb-1">Description</label>
        <textarea
          name="description"
          rows="3"
          value={product.description}
          onChange={handleChange}
          className={`w-full border p-3 rounded-lg focus:ring-2 focus:outline-none ${
            errors.description
              ? 'border-red-500 focus:ring-red-400'
              : 'border-gray-300 focus:ring-blue-400'
            }`}
          placeholder="Describe your product..."

```

```
    />
    {errors.description && <p className="text-red-500 text-sm mt-
1">{errors.description}</p>}
  </div>
```

```
  {/* Category & Subcategory */}
  <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
    <InputField
      label="Category"
      name="category"
      value={product.category}
      onChange={handleChange}
      error={errors.category}
    />
    <InputField
      label="Subcategory"
      name="subCategory"
      value={product.subCategory}
      onChange={handleChange}
      error={errors.subCategory}
    />
  </div>
```

```
  {/* Size & Stock */}
  <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
    {/*
      <InputField
        label="Available Sizes (comma-separated)"
        name="size"
        value={product.size}
        onChange={handleChange}
        error={errors.size}
      />
    */}
  </div>
```

```

<InputField
  label="Stock Quantity"
  name="stock"
  type="number"
  value={product.stock}
  onChange={handleChange}
  error={errors.stock}
/>
</div>

{/* Image Upload */}
<div
  className={`border p-4 rounded-lg transition ${
    errors.image ? 'border-red-400 bg-red-50' : 'border-dashed border-gray-300 bg-gray-50
hover:bg-gray-100'
  }`}
>
  <label className="block text-gray-600 font-medium mb-2">Upload Product
Image</label>
  <input
    type="file"
    name="image"
    onChange={handleImageChange}
    className="file:mr-3 file:py-2 file:px-4 file:rounded-md file:border-0 file:text-sm
file:font-semibold file:bg-blue-500 file:text-white hover:file:bg-blue-600"
  />
  {errors.image && <p className="text-red-500 text-sm mt-1">{errors.image}</p>}

  {imagePreview && (
    <div className="mt-3 text-center">
      <p className="text-gray-700 text-sm mb-1">Preview:</p>
      <img
        src={imagePreview}
        alt="Preview"
        className="h-40 w-40 object-cover rounded-lg mx-auto border shadow-sm"
      />
    </div>
  )}

```

```

    </div>
  )}
</div>

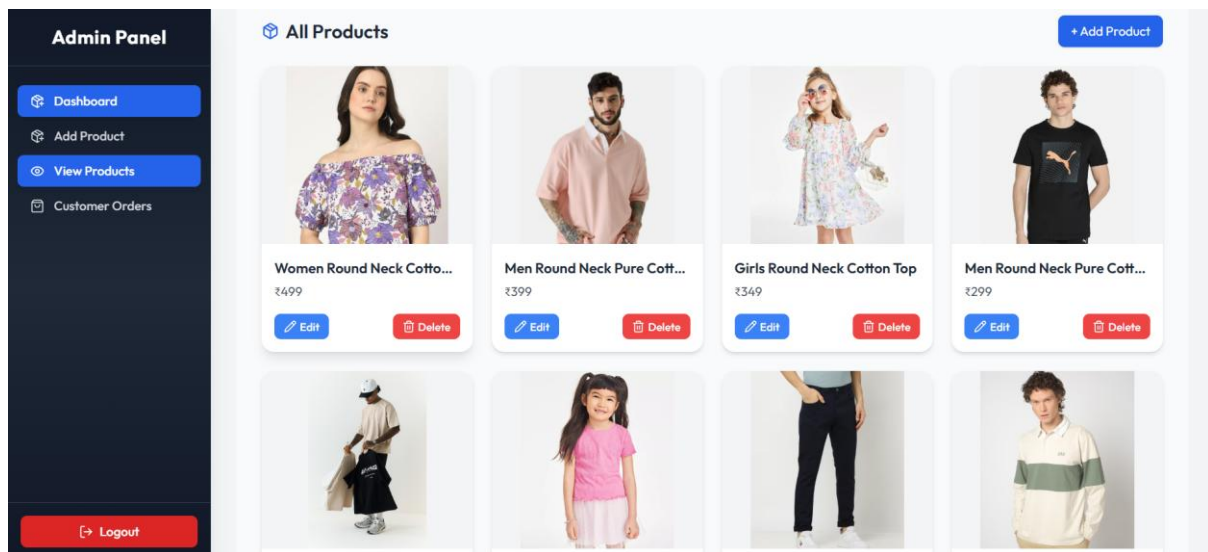
  { /* Submit Button */}
  <button
    type="submit"
    className="w-full bg-gradient-to-r from-blue-500 to-blue-600 text-white font-semibold
py-3 rounded-lg shadow-md hover:from-blue-600 hover:to-blue-700 transition-all"
    >
    Add Product
  </button>
</form>
</div>
</div>
);
};

/* Reusable Input Field Component */
const InputField = ({ label, name, type = 'text', value, onChange, error }) => (
  <div>
    <label className="block text-gray-700 font-medium mb-1">{label}</label>
    <input
      type={type}
      name={name}
      value={value}
      onChange={onChange}
      placeholder={label}
      className={`border p-3 rounded-lg w-full focus:ring-2 focus:outline-none ${
        error ? 'border-red-500 focus:ring-red-400' : 'border-gray-300 focus:ring-blue-400'
      }}`
    />
    {error && <p className="text-red-500 text-sm mt-1">{error}</p>}
  </div>
);

export default AddProduct;

```


ViewProduct.jsx :



```
import React, { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import { Pencil, Trash2, Package } from "lucide-react";
import api from "../api/api";
```

```
const ViewProducts = () => {
  const [products, setProducts] = useState([]);
  const navigate = useNavigate();

  // Fetch all products
  const fetchProducts = async () => {
    try {
      const res = await api.get("/admin/products");
      if (res.data.success && res.data.products) {
        setProducts(res.data.products);
      } else {
        setProducts([]);
      }
    } catch (err) {
      console.error("Error fetching products:", err);
    }
  };
};
```

```
useEffect(() => {  
  fetchProducts();  
}, []);
```

```
const deleteProduct = async (id) => {  
  const confirmDelete = window.confirm(  
    "Are you sure you want to delete this product?"  
  );  
  if (!confirmDelete) return;
```

```
  try {  
    const res = await api.delete(`/admin/products/${id}`);  
    if (res.data.success) {  
      setProducts((prev) => prev.filter((item) => item._id !== id));  
      alert("Product deleted successfully!");  
    } else {  
      alert(res.data.message || "Failed to delete product!");  
    }  
  } catch (err) {  
    console.error("Error deleting product:", err.response || err);  
    alert("Error deleting product");  
  }  
};
```

```
const editProduct = (id) => {  
  navigate(`/admin/product/edit/${id}`);  
};
```

```
return (  
  <div className="p-6 md:p-8 bg-gray-50 min-h-screen">  
    { /* Header */ }  
    <div className="flex items-center justify-between mb-6">  
      <h2 className="text-2xl font-semibold text-gray-800 flex items-center gap-2">  
        <Package size={22} className="text-blue-600" />  
        All Products
```

```

</h2>
<button
  onClick={() => navigate("/admin/add-product")}
  className="bg-blue-600 text-white px-4 py-2 rounded-lg shadow hover:bg-blue-700
transition"
  >
    + Add Product
</button>
</div>

{/* Product Grid —  4 per row on large screens */}
<div className="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 xl:grid-cols-4 gap-6">
  {products.length === 0 ? (
    <p className="text-gray-500 col-span-full text-center py-10">
      No products found.
    </p>
  ) : (
    products.map((item) => (
      <div
        key={item._id}
        className="bg-white rounded-2xl shadow-md hover:shadow-lg transition duration-300
overflow-hidden flex flex-col"
      >
        {/* Product Image */}
        <div className="relative w-full h-56 bg-gray-100 flex items-center justify-center
overflow-hidden">
          <img
            src={
              item.image
                ? `http://localhost:3000${item.image}`
                : "https://via.placeholder.com/300x300?text=No+Image"
            }
            alt={item.name}
            className="w-full h-full object-contain transition-transform duration-300
hover:scale-105"
          />

```

```

    </div>
    { /* Product Details */ }
    <div className="p-4 flex flex-col flex-grow">
      <h3 className="text-lg font-semibold text-gray-800 truncate">
        {item.name}
      </h3>
      <p className="text-gray-600 mt-1 flex-grow">
        ₹{item.price.toLocaleString()}
      </p>
      { /* Action Buttons */ }
      <div className="mt-4 flex justify-between">
        <button
          onClick={() => editProduct(item._id)}
          className="flex items-center gap-1 bg-blue-500 hover:bg-blue-600 text-white px-3
py-1.5 rounded-lg text-sm font-medium transition"
        >
          <Pencil size={16} /> Edit
        </button>
        <button
          onClick={() => deleteProduct(item._id)}
          className="flex items-center gap-1 bg-red-500 hover:bg-red-600 text-white px-3
py-1.5 rounded-lg text-sm font-medium transition"
        >
          <Trash2 size={16} /> Delete
        </button>
      </div>
    </div>
  </div>
))
})
</div>
</div>
);
};

export default ViewProducts;

```

EditProduct.jsx :

The screenshot shows a web application interface for editing a product. On the left is a dark sidebar titled 'Admin Panel' with links to 'Dashboard', 'Add Product', 'View Products', and 'Customer Orders'. A 'Logout' button is at the bottom. The main content area has a yellow header 'Edit Product' with the subtitle 'Update product details'. The form contains the following fields:

- Product Name: 'Women Round Neck Cotton Top' (with a price of '499' next to it)
- Description: 'A lightweight, usually knitted, pullover shirt, close-fitting and with a round neckline and short sleeves, worn as an undershirt or outer garment.'
- Category: 'Women' (with 'Topwear' as an option)
- Size: '50'
- Image Upload: A section with 'Upload Image', a yellow 'Choose file' button, and the text 'No file chosen'.
- Preview: A small image of a woman wearing a floral top.

```
import React, { useState, useEffect } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import api from '../api/api';
import { validateProductForm } from '../validation';
```

```
const EditProduct = () => {
  const { id } = useParams();
  const navigate = useNavigate();
```

```
  const [product, setProduct] = useState({
    name: "",
    description: "",
    price: "",
    category: "",
    subCategory: "",
    size: "",
    stock: "",
    image: null,
  });
```

```
  const [errors, setErrors] = useState({});
  const [imagePreview, setImagePreview] = useState(null);
```

```

// Fetch product details
useEffect(() => {
  const fetchProduct = async () => {
    try {
      const res = await api.get(`/api/product/${id}`);
      const data = res.data;

      console.log("Fetched sizes:", data); // Debug

      setProduct({
        name: data.name || "",
        description: data.description || "",
        price: data.price || "",
        category: data.category || "",
        subCategory: data.subCategory || "",
        size: Array.isArray(data.size) ? data.size.join(',') : data.size || "",
        stock: data.stock || "",
        image: null, // leave null initially
      });

      setImagePreview(data.image || null);

    } catch (err) {
      console.error(err);

      alert('❌ Failed to fetch product data');
    }
  };

  fetchProduct();
}, [id]);

const handleChange = (e) => {
  setProduct({ ...product, [e.target.name]: e.target.value });
  setErrors({ ...errors, [e.target.name]: "" });
};

```

```

const handleImageChange = (e) => {
  const file = e.target.files[0];
  if (file) {
    setProduct({ ...product, image: file });
    setImagePreview(URL.createObjectURL(file));
    setErrors({ ...errors, image: " " });
  }
};

```

```

const handleSubmit = async (e) => {
  e.preventDefault();

```

```

  const { isValid, errors: validationErrors } = validateProductForm(product);
  if (!isValid) {
    setErrors(validationErrors);
    console.log("validation error");
    return;
  }

```

```

  try {
    const formData = new FormData();
    formData.append('name', product.name);
    formData.append('description', product.description);
    formData.append('price', product.price);
    formData.append('category', product.category);
    formData.append('subCategory', product.subCategory);
    formData.append('stock', product.stock);
    // Convert comma-separated sizes to array
    /* formData.append(
      'size',
      JSON.stringify(product.size.split(',').map(s => s.trim()))
    ); */

```

```

    if (product.image) {
      formData.append('image', product.image);
    }

```

```

    }

    await api.put(`/api/product/${id}`, formData, {
      headers: { 'Content-Type': 'multipart/form-data' },
    });

    alert('✅ Product updated successfully!');
    navigate('/admin/view-products');
  } catch (err) {
    console.error(err.response || err);

    alert(err.response?.data?.message || '❌ Error updating product');
  }
};

return (
  <div className="min-h-screen flex items-center justify-center bg-gradient-to-br from-gray-50 to-gray-100 p-8">
    <div className="w-full max-w-2xl bg-white rounded-2xl shadow-lg border border-gray-100">
      <div className="bg-gradient-to-r from-yellow-500 to-yellow-600 text-white text-center py-4 rounded-t-2xl">
        <h2 className="text-2xl font-semibold tracking-wide">🛒 Edit Product</h2>
        <p className="text-sm opacity-90">Update product details</p>
      </div>

      <form onSubmit={handleSubmit} className="p-8 space-y-5">
        { /* Name & Price */ }

        <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
          <div>
            <input
              type="text"
              name="name"
              placeholder="Product Name"
              value={product.name}
              onChange={handleChange}

```



```

        className={`border p-3 rounded-lg w-full focus:ring-2 focus:outline-none ${
            errors.name ? 'border-red-500 focus:ring-red-400' : 'border-gray-300 focus:ring-
yellow-400'
        }}
    />

    {errors.name && <p className="text-red-500 text-sm mt-1">{errors.name}</p>}
</div>
<div>
    <input
        type="number"
        name="price"
        placeholder="Price"
        value={product.price}
        onChange={handleChange}
        className={`border p-3 rounded-lg w-full focus:ring-2 focus:outline-none ${
            errors.price ? 'border-red-500 focus:ring-red-400' : 'border-gray-300 focus:ring-
yellow-400'
        }}
    />

    {errors.price && <p className="text-red-500 text-sm mt-1">{errors.price}</p>}
</div>
</div>

{/* Description */}
<div>
    <textarea
        name="description"
        placeholder="Product Description"
        value={product.description}
        onChange={handleChange}
        rows="3"
        className={`border p-3 rounded-lg w-full focus:ring-2 focus:outline-none ${
            errors.description ? 'border-red-500 focus:ring-red-400' : 'border-gray-300 focus:ring-
yellow-400'
        }}
    />

```

```

        {errors.description && <p className="text-red-500 text-sm mt-
1">{errors.description}</p>}
    </div>

    {/* Category & Subcategory */}
    <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
        <div>
            <input
                type="text"
                name="category"
                placeholder="Category"
                value={product.category}
                onChange={handleChange}
                className={`border p-3 rounded-lg w-full focus:ring-2 focus:outline-none ${
                    errors.category ? 'border-red-500 focus:ring-red-400' : 'border-gray-300 focus:ring-
yellow-400'
                }}`
            />
            {errors.category && <p className="text-red-500 text-sm mt-
1">{errors.category}</p>}
        </div>
        <div>
            <input
                type="text"
                name="subCategory"
                placeholder="Subcategory"
                value={product.subCategory}
                onChange={handleChange}
                className={`border p-3 rounded-lg w-full focus:ring-2 focus:outline-none ${
                    errors.subCategory ? 'border-red-500 focus:ring-red-400' : 'border-gray-300
focus:ring-yellow-400'
                }}`
            />
            {errors.subCategory && <p className="text-red-500 text-sm mt-
1">{errors.subCategory}</p>}
        </div>

```

```
</div>
```

```
{/* Size & Stock */}
```

```
<div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
```

```
<div>
```

```
<input
```

```
  type="number"
```

```
  name="stock"
```

```
  placeholder="Stock Quantity"
```

```
  value={product.stock}
```

```
  onChange={handleChange}
```

```
  className={`border p-3 rounded-lg w-full focus:ring-2 focus:outline-none ${
```

```
    errors.stock ? 'border-red-500 focus:ring-red-400' : 'border-gray-300 focus:ring-  
yellow-400'
```

```
  }}`
```

```
{errors.stock && <p className="text-red-500 text-sm mt-1">{errors.stock}</p>}
```

```
</div>
```

```
</div>
```

```
{/* Image Upload */}
```

```
<div
```

```
  className={`flex flex-col sm:flex-row items-center gap-4 border p-4 rounded-lg  
transition ${
```

```
    errors.image ? 'border-red-400 bg-red-50' : 'border-dashed border-gray-300 bg-gray-50  
hover:bg-gray-100'
```

```
  }}`
```

```
>
```

```
<label className="text-gray-600 font-medium w-full sm:w-1/3 text-center sm:text-  
left">
```

```
  Upload Image
```

```
</label>
```

```
<input
```

```
  type="file"
```

```
  name="image"
```

```

        onChange={handleImageChange}
        className="w-full sm:w-2/3 file:mr-3 file:py-2 file:px-4 file:rounded-md file:border-0
file:text-sm file:font-semibold file:bg-yellow-500 file:text-white hover:file:bg-yellow-600"
    />
</div>
{errors.image && <p className="text-red-500 text-sm mt-1">{errors.image}</p>}

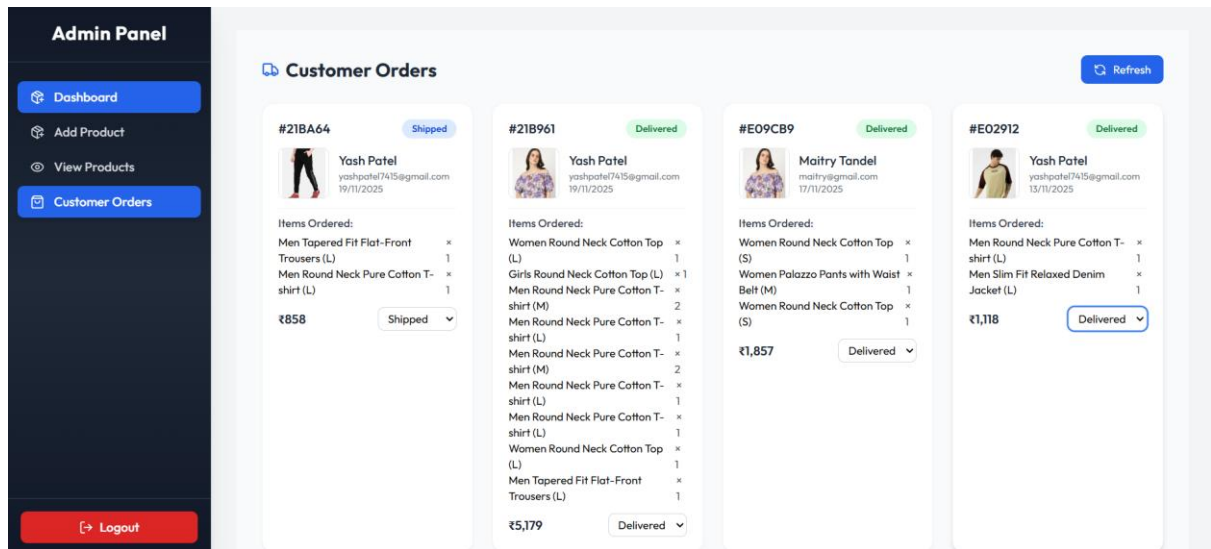
{/* Image Preview */}
{imagePreview && (
    <div className="mt-3 text-center">
        <p className="text-gray-700 text-sm mb-1">Preview:</p>
        <img src={imagePreview} alt="Preview" className="h-40 w-40 object-cover
rounded-lg mx-auto border" />
    </div>
)}

{/* Submit Button */}
<button
    type="submit"
    className="w-full bg-gradient-to-r from-yellow-500 to-yellow-600 text-white font-
semibold py-3 rounded-lg shadow-md hover:from-yellow-600 hover:to-yellow-700 transition"
    >
    Update Product
</button>
</form>
</div>
</div>
);
};

export default EditProduct;

```

CustomerOrders :



```
import React, { useEffect, useState } from "react";
import { Truck, Package, RefreshCcw } from "lucide-react";
import api from "../api/api";

const ViewOrders = () => {
  const [orders, setOrders] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  const statusOptions = ["Pending", "Shipped", "Delivered", "Cancelled"];

  useEffect(() => {
    fetchOrders();
  }, []);

  const fetchOrders = async () => {
    try {
      setLoading(true);
      setError("");
      const token = localStorage.getItem("token");
      if (!token) throw new Error("You are not logged in.");
```

```

const res = await api.get("/api/orders", {
  headers: { Authorization: `Bearer ${token}` },
});

if (res.data.success) setOrders(res.data.orders);
else setOrders([]);
} catch (err) {
  console.error("Error fetching orders:", err);
  setError(err.response?.data?.message || "Failed to fetch orders.");
} finally {
  setLoading(false);
}
};

const handleStatusChange = async (orderId, newStatus) => {
  try {
    const token = localStorage.getItem("token");
    if (!token) throw new Error("Not logged in.");

    const res = await api.put(
      `/api/orders/update/${orderId}`,
      { status: newStatus },
      { headers: { Authorization: `Bearer ${token}` } }
    );

    if (res.data.success) {
      setOrders((prev) =>
        prev.map((order) =>
          order._id === orderId ? { ...order, status: newStatus } : order
        )
      );
    }
  } catch (err) {
    console.error("Error updating status:", err);
    alert(err.response?.data?.message || "Failed to update status");
  }
}

```

```
};
```

```
if (loading)
```

```
  return (
```

```
    <div className="flex flex-col items-center justify-center min-h-[60vh] text-gray-500">
```

```
      <RefreshCcw size={30} className="animate-spin mb-2 text-blue-500" />
```

```
      <p>Loading orders...</p>
```

```
    </div>
```

```
  );
```

```
if (error)
```

```
  return (
```

```
    <div className="flex flex-col items-center justify-center min-h-[60vh] text-red-600 font-medium">
```

```
      <p>{error}</p>
```

```
    </div>
```

```
  );
```

```
if (orders.length === 0)
```

```
  return (
```

```
    <div className="flex flex-col items-center justify-center min-h-[60vh] text-gray-500">
```

```
      <Package size={64} className="text-gray-400 mb-3" />
```

```
      <p className="text-lg font-medium">No orders found yet.</p>
```

```
    </div>
```

```
  );
```

```
return (
```

```
  <div className="p-6 md:p-8 bg-gray-50 min-h-screen">
```

```
    <div className="flex items-center justify-between mb-6">
```

```
      <h2 className="text-2xl font-bold text-gray-800 flex items-center gap-2">
```

```
        <Truck size={22} className="text-blue-600" />
```

```
        Customer Orders
```

```
      </h2>
```

```
      <button
```

```
        onClick={fetchOrders}
```

```

        className="flex items-center gap-2 bg-blue-600 text-white text-sm px-4 py-2 rounded-lg
        hover:bg-blue-700 transition"
      >
        <RefreshCcw size={16} /> Refresh
      </button>
    </div>

```

```

    {/*  Now 4 boxes per row (responsive grid) */}
    <div className="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 xl:grid-cols-4 gap-6">
      {orders.map((order) => (
        <div
          key={order._id}
          className="bg-white rounded-2xl shadow-sm border border-gray-100 hover:shadow-
md transition p-5"
        >

```

```

          <div className="flex items-center justify-between mb-3">
            <p className="font-semibold text-gray-800">
              #{order._id.slice(-6).toUpperCase()}
            </p>
            <span
              className={`px-3 py-1 text-xs font-semibold rounded-full ${
                order.status === "Delivered"
                  ? "bg-green-100 text-green-700"
                  : order.status === "Shipped"
                  ? "bg-blue-100 text-blue-700"
                  : order.status === "Cancelled"
                  ? "bg-red-100 text-red-700"
                  : "bg-yellow-100 text-yellow-700"
              }`}
            >

```

```

              {order.status}
            </span>
          </div>

```

```

    <div className="flex gap-3 items-center mb-3">
      <img

```



```

src={
  order.items?.[0]?.productId?.image
  ? `http://localhost:3000${order.items[0].productId.image}`
  : "https://via.placeholder.com/60x60?text=No+Image"
}
alt={order.items?.[0]?.productId?.name || "Product"}
className="w-16 h-16 rounded-lg border object-contain"
/>
<div>
  <p className="font-medium text-gray-800">
    {order.customerName || "Guest"}
  </p>
  <p className="text-xs text-gray-500">{order.email}</p>
  <p className="text-xs text-gray-500">
    {new Date(order.createdAt).toLocaleDateString()}
  </p>
</div>
</div>

<div className="mb-3 border-t border-gray-100 pt-2">
  <p className="text-sm text-gray-700 font-medium mb-1">
    Items Ordered:
  </p>
  {order.items?.map((item, i) => (
    <div key={i} className="flex justify-between text-sm">
      <span>
        {item.name} ({item.size || ""})
      </span>
      <span className="text-gray-600">× {item.qty}</span>
    </div>
  ))}
</div>

<div className="flex items-center justify-between mt-3">
  <p className="text-gray-800 font-semibold">
    ₹{order.totalAmount?.toLocaleString() || "0"}
  </p>
</div>

```

```

    </p>
    <select
      className="border border-gray-300 rounded-lg px-2 py-1 text-sm focus:ring-2
focus:ring-blue-500 outline-none"
      value={order.status}
      onChange={(e) => handleStatusChange(order._id, e.target.value)}
    >
      {statusOptions.map((status) => (
        <option key={status} value={status}>
          {status}
        </option>
      ))}
    </select>
  </div>
</div>
  )}
</div>
</div>
);
};

```

```

export default ViewOrders;

```

Backend :

Config :

db.js :

```
//import mongoose
const mongoose = require("mongoose")

//function to connect mongoose
const connDB = async() => {
  try{
    await mongoose.connect('mongodb+srv://yash25:8778@cluster0.pdjmynf.mongodb.net/E-Commerce')
    console.log("Mongoose Connected..!")
  } catch(err) {
    console.log("Mongoose Connection Error..!", err.message)
  }
}

module.exports = connDB;
```

Controller :

AdminController.js :

```
const Product = require("../models/Product");

// Create product
exports.createProduct = async (req, res) => {
  try {
    const { name, category, subCategory, price, stock, description } = req.body;
    const image = req.file ? `/uploads/${req.file.filename}` : null;

    if (!name || !category || !subCategory || !price || !stock) {
      return res.status(400).json({ success: false, message: "Missing required fields" });
    }

    const newProduct = new Product({ name, category, subCategory, price, stock, description,
    image });
    const savedProduct = await newProduct.save();
    res.status(201).json({ success: true, product: savedProduct });
  } catch (err) {
    console.error(err);
    res.status(500).json({ success: false, message: "Server error" });
  }
};

// Get all products
exports.getProducts = async (req, res) => {
  try {
    const products = await Product.find();
    res.status(200).json({ success: true, products });
  } catch (err) {
```

```
    console.error(err);
    res.status(500).json({ success: false, message: "Server error" });
  }
};
```

// Delete product

```
exports.deleteProduct = async (req, res) => {
  try {
    const { id } = req.params;
    console.log("Deleting product ID:", id); // Debug

    const deleted = await Product.findByIdAndDelete(id);
    if (!deleted) return res.status(404).json({ success: false, message: "Product not found" });

    res.json({ success: true, message: "Product deleted successfully" });
  } catch (err) {
    console.error(err);
    res.status(500).json({ success: false, message: "Server error" });
  }
};
```

AdminDashboardController.js :

```
const User = require("../models/Registration");
const Order = require("../models/Order");
const Product = require("../models/Product");

/**
 * Get total registered users
 */
exports.getTotalUsers = async (req, res) => {
  try {
    const count = await User.countDocuments();
    res.status(200).json({ success: true, count });
  } catch (err) {
    console.error("Error fetching total users:", err);
    res.status(500).json({ success: false, message: "Server error" });
  }
};

/**
 * Get total orders
 */
exports.getTotalOrders = async (req, res) => {
  try {
    const count = await Order.countDocuments();
    res.status(200).json({ success: true, count });
  } catch (err) {
    console.error("Error fetching total orders:", err);
    res.status(500).json({ success: false, message: "Server error" });
  }
};

/**
 * Get total products
 */
```

```

exports.getTotalProducts = async (req, res) => {
  try {
    const count = await Product.countDocuments();
    res.status(200).json({ success: true, count });
  } catch (err) {
    console.error("Error fetching total products:", err);
    res.status(500).json({ success: false, message: "Server error" });
  }
};

```

```

/**
 * Get combined dashboard stats (users, orders, products)
 */
exports.getDashboardStats = async (req, res) => {
  try {
    const [users, orders, products] = await Promise.all([
      User.countDocuments(),
      Order.countDocuments(),
      Product.countDocuments(),
    ]);

    res.status(200).json({
      success: true,
      stats: {
        totalUsers: users,
        totalOrders: orders,
        totalProducts: products,
      },
    });
  } catch (err) {
    console.error("Error fetching dashboard stats:", err);
    res.status(500).json({ success: false, message: "Server error" });
  }
};

```

```

* Get orders count grouped by date (for daily sales chart)
*/
exports.getOrdersByDate = async (req, res) => {
  try {
    const orders = await Order.aggregate([
      {
        $group: {
          _id: {
            $dateToString: { format: "%Y-%m-%d", date: "$createdAt" },
          },
          count: { $sum: 1 },
        },
      },
      { $sort: { _id: 1 } },
    ]);

    const formatted = orders.map((o) => ({
      date: o._id,
      count: o.count,
    }));

    res.json(formatted);
  } catch (err) {
    console.error("Error fetching orders by date:", err);
    res.status(500).json({ message: "Server error" });
  }
};

/**
* Get total sales grouped by month (for monthly sales chart)
*/
exports.getMonthlySales = async (req, res) => {
  try {
    const sales = await Order.aggregate([
      {
        $group: {

```



```
    _id: { $month: "$createdAt" },
    totalSales: { $sum: "$totalAmount" },
  },
},
{ $sort: { "_id": 1 } },
]);
```

```
const monthNames = [
  "Jan", "Feb", "Mar", "Apr", "May", "Jun",
  "Jul", "Aug", "Sep", "Oct", "Nov", "Dec",
];
```

```
const monthlySales = monthNames.map((name, i) => {
  const found = sales.find((s) => s._id === i + 1);
  return { month: name, total: found ? found.totalSales : 0 };
});
```

```
res.json(monthlySales);
} catch (error) {
  console.error("Error fetching monthly sales:", error);
  res.status(500).json({ message: "Server error" });
}
};
```

Authcontroller.js :

```
const jwt = require("jsonwebtoken");
const bcrypt = require("bcryptjs");
const User = require("../models/Registration");
const sendEmail = require("../Utils/sendEmail");

// ===== Forgot Password =====
exports.forgotPassword = async (req, res) => {
  try {
    const { email } = req.body;

    const user = await User.findOne({ email });
    if (!user) return res.status(404).json({ message: "User not found" });

    // Generate a reset token valid for 15 minutes
    const resetToken = jwt.sign(
      { id: user._id },
      process.env.JWT_SECRET,
      { expiresIn: "15m" }
    );

    // Send reset link to user's email
    const resetLink = `http://localhost:5173/reset-password/${resetToken}`;
    await sendEmail(
      user.email,
      "Password Reset Request",
      `
      <h2>Password Reset</h2>
      <p>Click the link below to reset your password. This link expires in 15 minutes.</p>
      <a href="${resetLink}" target="_blank">${resetLink}</a>
      `
    );

    res.json({ message: "Password reset email sent successfully" });
  }
}
```

```
    } catch (error) {  
      console.error(error);  
      res.status(500).json({ message: "Error sending password reset email" });  
    }  
  };  
};
```

```
// ===== Reset Password =====
```

```
exports.resetPassword = async (req, res) => {  
  try {  
    const { token } = req.params;  
    const { newPassword } = req.body;  
  
    // Verify token  
    const decoded = jwt.verify(token, process.env.JWT_SECRET);  
    const user = await User.findById(decoded.id);  
    if (!user) return res.status(404).json({ message: "User not found" });  
  
    user.password = newPassword;  
    await user.save();  
  
    res.json({ message: "Password reset successful" });  
  } catch (error) {  
    console.error(error);  
    res.status(400).json({ message: "Invalid or expired token" });  
  }  
};
```

CartController.js :

```
const Cart = require("../models/Cart");

// Get cart for a user
exports.getCart = async (req, res) => {
  try {
    const cart = await Cart.findOne({ user: req.user._id }).populate("items.product");
    if (!cart) return res.status(200).json({ success: true, items: [] });
    res.status(200).json({ success: true, items: cart.items });
  } catch (err) {
    console.error(err);
    res.status(500).json({ success: false, message: "Server error" });
  }
};

// Add/update item in cart
exports.addToCart = async (req, res) => {
  const { productId, size, quantity } = req.body;

  if (!productId || !size || !quantity)
    return res.status(400).json({ success: false, message: "All fields are required" });

  try {
    let cart = await Cart.findOne({ user: req.user._id });

    if (!cart) {
      cart = new Cart({ user: req.user._id, items: [{ product: productId, size, quantity }] });
    } else {
      const itemIndex = cart.items.findIndex(
        item => item.product.toString() === productId && item.size === size
      );

      if (itemIndex > -1) {
        cart.items[itemIndex].quantity += quantity;
      }
    }
  }
};
```

```

    } else {
      cart.items.push( { product: productId, size, quantity } );
    }
  }

  await cart.save();
  res.status(200).json( { success: true, cart } );
} catch (err) {
  console.error(err);
  res.status(500).json( { success: false, message: "Server error" } );
}
};

// Update quantity
exports.updateQuantity = async (req, res) => {
  const { productId, size, quantity } = req.body;

  try {
    const cart = await Cart.findOne( { user: req.user._id } );
    if (!cart) return res.status(404).json( { success: false, message: "Cart not found" } );

    const itemIndex = cart.items.findIndex(
      item => item.product.toString() === productId && item.size === size
    );

    if (itemIndex === -1) return res.status(404).json( { success: false, message: "Item not found"
    });

    if (quantity <= 0) {
      cart.items.splice(itemIndex, 1);
    } else {
      cart.items[itemIndex].quantity = quantity;
    }

    await cart.save();
    res.status(200).json( { success: true, cart } );
  }

```

```

    } catch (err) {
      console.error(err);
      res.status(500).json({ success: false, message: "Server error" });
    }
  };

// Remove item from cart
exports.removeFromCart = async (req, res) => {
  const { productId, size } = req.body;

  try {
    const cart = await Cart.findOne({ user: req.user._id });
    if (!cart) return res.status(404).json({ success: false, message: "Cart not found" });

    cart.items = cart.items.filter(
      item => !(item.product.toString() === productId && item.size === size)
    );

    await cart.save();
    res.status(200).json({ success: true, cart });
  } catch (err) {
    console.error(err);
    res.status(500).json({ success: false, message: "Server error" });
  }
};

```

OrderController.js :

```
const Order = require("../models/Order");
```

```
const createOrder = async (req, res) => {
```

```
  try {
```

```
    const {
```

```
      items,
```

```
      totalAmount,
```

```
      address, // required
```

```
      phone,
```

```
      city,
```

```
      state,
```

```
      zip,
```

```
      paymentMethod
```

```
    } = req.body;
```

```
    // Make sure address is provided
```

```
    if (!address) {
```

```
      return res.status(400).json({ success: false, message: "Address is required" });
```

```
    }
```

```
    const newOrder = await Order.create({
```

```
      customerName: req.user.name,
```

```
      email: req.user.email,
```

```
      phone,
```

```
      address,
```

```
      city,
```

```
      state,
```

```
    zip,  
    paymentMethod,  
    items,  
    totalAmount,  
    status: "Pending",  
  });
```

```
    res.status(201).json({ success: true, order: newOrder });  
  } catch (err) {  
    console.error(err);  
    res.status(500).json({ success: false, message: "Server error" });  
  }  
};
```

```
// Get orders (admin = all, user = own)  
  
/* const getOrders = async (req, res) => {  
  try {  
    const orders = await Order.find().sort({ createdAt: -1 });  
    res.json({ success: true, orders });  
  } catch (err) {  
    console.error(err);  
    res.status(500).json({ success: false, message: "Server error" });  
  }  
}; */
```

```
// Get orders (admin = all, user = own)  
  
const getOrders = async (req, res) => {  
  try {  
    let ordersQuery;
```



```

if (req.user.role === "admin") {

  // Admin → all orders

  ordersQuery = Order.find();

} else {

  // User → only their orders

  ordersQuery = Order.find({ email: req.user.email });

}


// Populate product details (name, image, price)

const orders = await ordersQuery

  .populate("items.productId", "name image price")

  .sort({ createdAt: -1 });

res.json({ success: true, orders });

} catch (err) {

  console.error(err);

  res.status(500).json({ success: false, message: "Server error" });

}

};

```

```

// Update order status (admin only)

//  Update Order Status Controller (Admin Only)

const updateOrderStatus = async (req, res) => {

  try {

    const { orderId } = req.params;

    const { status } = req.body;

```

```
const order = await Order.findById(orderId);

if (!order) return res.status(404).json({ message: "Order not found" });


order.status = status;

await order.save();


res.json({ success: true, order });
} catch (err) {
  console.error(err);
  res.status(500).json({ success: false, message: "Server error" });
}
};


module.exports = { createOrder, getOrders, updateOrderStatus };
```

ProductController.js :

```
const Product = require("../models/Product");
```

```
// Create a new product
```

```
exports.createProduct = async (req, res) => {
```

```
  try {
```

```
    const { name, description, price, category, subCategory, size, stock } = req.body;
```

```
    //const sizes = JSON.parse(size); // from frontend JSON
```

```
    const imageUrl = req.file ? `/uploads/${req.file.filename}` : null;
```

```
    const newProduct = new Product({
```

```
      name,
```

```
      description,
```

```
      price,
```

```
      category,
```

```
      subCategory,
```

```
      //size: sizes,
```

```
      stock,
```

```
      image: imageUrl
```

```
    });
```

```
    await newProduct.save();
```

```
    res.status(201).json({ message: "Product added successfully", product: newProduct });
```

```
  } catch (err) {
```

```
    console.error(err);
```

```
    res.status(500).json({ message: "Error adding product", error: err.message });  
  
  }  
  
};
```

```
// Get all products
```

```
exports.getProducts = async (req, res) => {  
  
  try {  
  
    const products = await Product.find();  
  
    res.status(200).json({ success: true, products });  
  
  } catch (err) {  
  
    res.status(500).json({ success: false, message: "Server error" });  
  
  }  
  
};
```

```
// Get product by ID
```

```
exports.getProductById = async (req, res) => {  
  
  try {  
  
    const product = await Product.findById(req.params.id);  
  
    if (!product) return res.status(404).json({ success: false, message: "Product not found" });  
  
    console.log("Fetched product:", product); // <— add this
```

```
// Build full URL for the image
```

```
const imageUrl = product.image  
  
  ? `${req.protocol}://${req.get('host')}${product.image}`  
  
  : null;
```

```

    res.status(200).json({
      ...product.toObject(),
      image: imageUrl,
    });
  } catch (err) {
    res.status(500).json({ success: false, message: "Server error" });
  }
};

```

```

exports.updateProduct = async (req, res) => {
  try {
    const product = await Product.findById(req.params.id);

    if (!product) return res.status(404).json({ message: 'Product not found' });

    // Update fields from req.body
    if (req.body.name) product.name = req.body.name;
    if (req.body.description) product.description = req.body.description;
    if (req.body.category) product.category = req.body.category;
    if (req.body.subCategory) product.subCategory = req.body.subCategory;
    if (req.body.price) product.price = Number(req.body.price);
    if (req.body.stock) product.stock = Number(req.body.stock);
    if (req.body.size) product.size = JSON.parse(req.body.size); // array

    // Update image if new one uploaded
    if (req.file) {
      product.image = `/uploads/${req.file.filename}`;
    }
  }
};

```

```
}
```

```
await product.save();
```

```
res.json({ message: 'Product updated successfully', product });
```

```
} catch (err) {
```

```
  console.error(err);
```

```
  res.status(500).json({ message: 'Server error', error: err.message });
```

```
}
```

```
};
```

UserController.js :

```
const regisModel = require("../models/Registration");
```

```
const jwt = require("jsonwebtoken");
```

```
const nodemailer = require("nodemailer");
```

```
const bcrypt = require("bcrypt");
```

```
// Generate JWT token
```

```
const generateToken = (id) => {
```

```
  return jwt.sign({ id }, process.env.JWT_SECRET, { expiresIn: "7d" });
```

```
};
```

```
// Get users
```

```
const getuser = async (req, res) => {
```

```
  try {
```

```
    const { id, name, email, phone } = req.query;
```

```
    const query = {
```

```
      ...(id ? { _id: id } : {}),
```

```
      ...(name ? { name } : {}),
```

```
      ...(email ? { email } : {}),
```

```
      ...(phone ? { phone } : {}),
```

```
    };
```

```
    const result = await regisModel.find(query, "_id name email phone role");
```

```
    res.json(result);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: "Server Error", err });
  }
};
```

// Register user

```
const adduser = async (req, res) => {
  try {
    const { name, email, phone, password, role } = req.body;

    if (!name || !email || !phone || !password) {
      return res.status(400).json({ msg: "All fields are required." });
    }

    if (!/^[^s@]+@[^s@]+\.[^s@]+$/.test(email)) {
      return res.status(400).json({ msg: "Invalid Email format." });
    }

    if (!/^[6-9]\d{9}$/.test(phone)) {
      return res.status(400).json({ msg: "Invalid Contact Number." });
    }

    if (password.length < 6 || password.length > 15) {
      return res.status(400).json({ msg: "Password must be 6-15 characters." });
    }
  }
};
```



```
}
```

```
const existingUser = await regisModel.findOne({ email });
```

```
if (existingUser) return res.status(400).json({ msg: "Email already registered." });
```

```
const newUser = await regisModel.create({
```

```
  name,
```

```
  email,
```

```
  phone,
```

```
  password,
```

```
  role,
```

```
});
```

```
const token = generateToken(newUser._id); // Generate JWT on registration
```

```
res.json({
```

```
  msg: "New User Added!",
```

```
  token,
```

```
  user: {
```

```
    id: newUser._id,
```

```
    name: newUser.name,
```

```
    email: newUser.email,
```

```
    phone: newUser.phone,
```

```
    role: newUser.role,
```

```
  },
```

```
});
```

```
} catch (err) {  
  
  console.error(err);  
  
  res.status(500).json({ error: "Server Error", err });  
  
}  
};  
  
// Login user  
  
const loginUser = async (req, res) => {  
  
  try {  
  
    const { email, password } = req.body;  
  
  
    if (!email || !password) {  
  
      return res.status(400).json({ msg: "Email and Password are required." });  
  
    }  
  
  
    const user = await regisModel.findOne({ email });  
  
    if (!user) return res.status(400).json({ msg: "User not found." });  
  
  
    const isMatch = await user.comparePassword(password);  
  
    if (!isMatch) return res.status(400).json({ msg: "Invalid password." });  
  
  
    const token = generateToken(user._id); // JWT token  
  
  
    res.json({  
  
      msg: "Login successful",  
  
      token,  

```

```
    user: {  
      id: user._id,  
      name: user.name,  
      email: user.email,  
      phone: user.phone,  
      role: user.role,  
    },  
  });  
} catch (err) {  
  console.error(err);  
  res.status(500).json({ error: "Server Error", err });  
}  
};
```

```
module.exports = { getUser, adduser, loginUser };
```

Middleware :

Admin.js :

```
const adminMiddleware = (req, res, next) => {  
  
  if (!req.user) {  
  
    return res.status(401).json({ message: "Unauthorized: No user logged in" });  
  
  }  
  
  
  if (!req.user.isAdmin) {  
  
    return res.status(403).json({ message: "Forbidden: Admins only" });  
  
  }  
  
  
  next();  
  
};  
  
  
module.exports = adminMiddleware;
```

authMiddleware.js :

```
// middlewares/authMiddleware.js  
  
const jwt = require("jsonwebtoken");  
  
const regisModel = require("../models/Registration");  
  
  
const protect = async (req, res, next) => {  
  
  let token;
```

```
if (req.headers.authorization && req.headers.authorization.startsWith("Bearer")) {  
  
  try {  
  
    // Extract token  
  
    token = req.headers.authorization.split(" ")[1];  
  
  
    // Verify token  
  
    const decoded = jwt.verify(token, process.env.JWT_SECRET);  
  
  
    // Find user and attach to request  
  
    req.user = await regisModel.findById(decoded.id).select("-password");  
  
  
    if (!req.user) {  
  
      return res.status(401).json({ success: false, message: "User not found" });  
  
    }  
  
  
    next();  
  
  } catch (err) {  
  
    console.error("JWT Error:", err);  
  
    return res.status(401).json({ success: false, message: "Token invalid" });  
  
  }  
  
}  
  
  
if (!token) {  
  
  return res.status(401).json({ success: false, message: "No token provided" });  
  
}  
  
};
```

```
//  Middleware for Admin-only routes

const adminOnly = (req, res, next) => {

  if (req.user && req.user.role === "admin") {

    next();

  } else {

    return res.status(403).json({ success: false, message: "Access denied. Admins only." });

  }

};

module.exports = { protect, adminOnly };
```

Uploads.js :

```
const multer = require("multer");

const path = require("path");

const fs = require("fs");

// Ensure uploads folder exists

const uploadsDir = path.join(__dirname, "../uploads");

if (!fs.existsSync(uploadsDir)) {

  fs.mkdirSync(uploadsDir, { recursive: true });

  console.log("Uploads folder created!");

}

// Storage configuration

const storage = multer.diskStorage({
```

```
destination: function (req, file, cb) {

  cb(null, uploadsDir); // Save in uploads folder

},

filename: function (req, file, cb) {

  const uniqueSuffix = Date.now() + "-" + Math.round(Math.random() * 1e9);

  const ext = path.extname(file.originalname);

  cb(null, file.fieldname + "-" + uniqueSuffix + ext);

},

});

// File filter for images only

const fileFilter = (req, file, cb) => {

  if (file.mimetype.startsWith("image/")) {

    cb(null, true);

  } else {

    cb(new Error("Only image files are allowed!"), false);

  }

};

// Max file size 5MB

const limits = {

  fileSize: 5 * 1024 * 1024,

};

// Upload instance

const upload = multer({ storage, fileFilter, limits });

module.exports = upload;
```

Models :

Cart.js :

```
const mongoose = require("mongoose");
```

```
const cartSchema = new mongoose.Schema({  
  user: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },  
  items: [  
    {  
      product: { type: mongoose.Schema.Types.ObjectId, ref: "Product", required: true },  
      size: { type: String, required: true },  
      quantity: { type: Number, default: 1 },  
    }  
  ]  
}, { timestamps: true });
```

```
module.exports = mongoose.model("Cart", cartSchema);
```


Order.js :

```
const mongoose = require('mongoose');

const orderSchema = new mongoose.Schema({

  customerName: { type: String, required: true },

  email: { type: String },

  phone: { type: String },

  address: { type: String, required: true },

  city: { type: String },

  state: { type: String },

  zip: { type: String },

  paymentMethod: { type: String, enum: ['cod', 'card', 'upi'], default: 'cod' },

  items: [

    {

      productId: { type: mongoose.Schema.Types.ObjectId, ref: 'Product', required: true },

      name: { type: String, required: true },

      price: { type: Number, required: true },

      qty: { type: Number, required: true },

      size: { type: String },

    }

  ],

  totalAmount: { type: Number, required: true },

  status: { type: String, enum: ['Pending', 'Shipped', 'Delivered', 'Cancelled'], default: 'Pending' },

}, { timestamps: true });

module.exports = mongoose.model('Order', orderSchema);
```

Product.js :

```
const mongoose = require("mongoose");

const productSchema = new mongoose.Schema({
  name: { type: String, required: true },
  category: { type: String, required: true },
  subCategory: { type: String, required: true },
  price: { type: Number, required: true },
  stock: { type: Number, required: true },
  size : { type : String, require : true},
  description: { type: String },
  image: { type: String } // URL or path to image
}, { timestamps: true });

module.exports = mongoose.model("Product", productSchema);
```

Registration.js :

```
const mongoose = require('mongoose');
const bcrypt = require('bcrypt');

const regisSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
```

```
      trim: true
    },
    email: {
      type: String,
      required: true,
      unique: true,
      lowercase: true,
      trim: true
    },
    phone: {
      type: String,
      required: true,
      trim: true
    },
    password: {
      type: String,
      required: true
    },
    role: {
      type: String,
      enum: ["user", "admin"],
      default: "user"
    },
    createdAt: {
      type: Date,
      default: Date.now
    }
  }
}
```

```

    }
  });

// Pre-save hook to hash password
regisSchema.pre('save', async function(next) {
  if (!this.isModified('password')) return next();

  try {
    const salt = await bcrypt.genSalt(10);

    this.password = await bcrypt.hash(this.password, salt);

    next();
  } catch (err) {
    next(err);
  }
});

// Method to compare password during login
regisSchema.methods.comparePassword = async function(plainPassword) {
  return await bcrypt.compare(plainPassword, this.password);
};

const regisModel = mongoose.model("user", regisSchema, "Register");

module.exports = regisModel;

```

Routes :

adminDashboard.js :

```
const express = require("express");

const router = express.Router();

const {
  getTotalUsers,
  getTotalOrders,
  getTotalProducts,
  getDashboardStats,
  getMonthlySales,
  getOrdersByDate,
} = require("../controller/AdminDashboardController");

// Individual count routes

router.get("/users-count", getTotalUsers);

router.get("/orders-count", getTotalOrders);

router.get("/products-count", getTotalProducts);

// Chart data routes

router.get("/monthly-sales", getMonthlySales);

router.get("/orders-by-date", getOrdersByDate);

// Combined stats route

router.get("/stats", getDashboardStats);

module.exports = router;
```

adminroutes.js :

```
const express = require("express");

const router = express.Router();

const multer = require("multer");

const path = require("path");

const { createProduct, getProducts, deleteProduct } = require("../controller/AdminController");

// Multer config

const storage = multer.diskStorage({

  destination: (req, file, cb) => cb(null, "uploads/"),

  filename: (req, file, cb) => cb(null, Date.now() + path.extname(file.originalname)),

});

const upload = multer( { storage } );

router.post("/products", upload.single("image"), createProduct);

router.get("/products", getProducts);

router.delete("/products/:id", deleteProduct); // <-- simple route

module.exports = router;
```

Authroutes.js :

```
const express = require('express');

const { forgotPassword, resetPassword } = require('../controller/authController');

const router = express.Router();

router.post("/forgot-password", forgotPassword);

router.post("/reset-password/:token", resetPassword);

module.exports = router
```

CartRoutes.js :

```
const express = require("express");

const router = express.Router();

const {
  getCart,
  addToCart,
  updateQuantity,
  removeFromCart,
} = require("../controller/CartController");

const { protect } = require("../middlewares/authMiddleware"); // JWT auth middleware

router.get("/", protect, getCart);

router.post("/add", protect, addToCart);

router.put("/update", protect, updateQuantity);

router.delete("/remove", protect, removeFromCart);

module.exports = router;
```

OrderRoutes.js :

```
const express = require("express");

const {
  createOrder,
  getOrders,
  updateOrderStatus,
} = require("../controller/orderController");

const { protect, adminOnly } = require("../middlewares/authMiddleware");

const router = express.Router();

//  Create order (any logged-in user)
router.post("/", protect, createOrder);

//  Get orders (user sees own orders, admin sees all)
router.get("/", protect, getOrders);

//  Update order status (admins only)
router.put("/update/:orderId", protect, adminOnly, updateOrderStatus);

module.exports = router;
```


Product.js :

```
const express = require("express");

const router = express.Router();

const {
  createProduct,
  getProducts,
  getProductById,
  updateProduct,
} = require("../controller/productController");

const upload = require("../middlewares/uploads");

//  Create a new product (with image upload)
router.post("/", upload.single("image"), createProduct);

//  Get all products
router.get("/", getProducts);

//  Get product by ID
router.get("/:id", getProductById);

//  Update product (with optional new image)
router.put("/:id", upload.single("image"), updateProduct);

module.exports = router;
```

UserRoutes.js :

```
const express = require ("express")
```

```
const { getuser, adduser,loginUser } = require ("../controller/UserController")
```

```
const app = express.Router()
```

```
app.get("/user", getuser)
```

```
app.post("/add", adduser)
```

```
app.post("/login",loginUser)
```

```
module.exports = app
```

Utils :

sendEmail.js :

```
const nodemailer = require("nodemailer");

const sendEmail = async (to, subject, html) => {

  const transporter = nodemailer.createTransport({

    service: "gmail",

    auth: {

      user: process.env.EMAIL_USER,

      pass: process.env.EMAIL_PASS,

    },

  });

  await transporter.sendMail({

    from: `"Wardrobe X" <${process.env.EMAIL_USER}>`,

    to,

    subject,

    html,

  });

};

module.exports = sendEmail;
```

index.js :

```
require('dotenv').config();

const express = require('express');

const path = require('path');

const fs = require('fs'); // <-- needed for Step 4

const userRoutes = require("./routes/UserRoutes");

const productRoutes = require("./routes/Products");

const adminRoutes = require("./routes/adminroutes");

const cartRoutes = require("./routes/CartRoutes");

const connDB = require('./config/db');

const cors = require("cors");

const orderRoutes = require ("./routes/OrderRoutes")

const adminDashboardRoutes = require("./routes/admindashboardroutes");

const authRoutes = require("./routes/authroutes");


const app = express();

app.use(express.json());


// Connect backend with frontend

app.use(cors({

  origin: "http://localhost:5173",

  credentials: true

}));
```

```
// Connect to MongoDB

connDB();

// ----- STEP 4: Ensure uploads folder exists -----

const uploadsDir = path.join(__dirname, 'uploads');

if (!fs.existsSync(uploadsDir)) {

  fs.mkdirSync(uploadsDir);

  console.log("Uploads folder created!");

}

// -----

// Serve uploaded images statically

app.use('/uploads', express.static(uploadsDir));

app.use("/uploads", express.static("uploads"));

// Routes

app.use("/api/admin", adminRoutes);

app.use("/", userRoutes);

app.use("/api/product", productRoutes);

app.use("/admin", adminRoutes);

app.use("/api/cart", cartRoutes)

app.use("/api/orders", orderRoutes)

app.use("/api/auth", authRoutes);

app.use("/admin", adminDashboardRoutes);

app.listen(3000, () => {
```

```
    console.log("Server is running on port 3000");  
  });
```